ОСНОВЫ ОБЛАЧНЫХ ТЕХНОЛОГИЙ

Занятие 4
ВИРТУАЛИЗАЦИЯ И ВЫЧИСЛИТЕЛЬНЫЕ РЕСУРСЫ ОБЛАКА

Теоретические сведения, разобранные примеры,
контрольные вопросы и лабораторная работа № 4

Рабочая среда. Все задания выполняются на собственном компьютере под управлением Windows, macOS или Linux, без учётной записи AWS и без каких-либо затрат. Программы на C++ собираются внутри контейнеров Docker, поэтому команды одинаковы во всех операционных системах. В Windows команды вводятся в терминале WSL (Ubuntu).

Содержание

1. Цель занятия и требования к подготовке

2. Рабочая среда

- 2.1. Проверка аппаратной поддержки виртуализации
- 2.2. VirtualBox и образ Ubuntu
- 2.3. Эмулятор AWS
- 2.4. draw.io

3. Виртуализация: теоретические основы

- 3.1. Понятие виртуальной машины
- 3.2. Требования Попека — Голдберга
- 3.3. Проблема архитектуры x86 и способы её решения
- 3.4. Виртуализация памяти и ввода-вывода
- 3.5. ВМ и контейнер

4. Гипервизоры

- 4.1. Первый и второй типы
- 4.2. KVM, VirtualBox, VMware
- 4.3. Сеть виртуальной машины

5. Вычислительные ресурсы облака: Amazon EC2

- 5.1. Система Nitro
- 5.2. Образы, инстансы, жизненный цикл
- 5.3. Семейства инстансов и vCPU
- 5.4. Хранилище инстанса
- 5.5. Пользовательские данные
- 5.6. Регионы, зоны доступности и размещение
- 5.7. Образы машин и воспроизводимость среды
- 5.8. Ограничения учётной записи

6. Блочное хранилище: тома EBS и локальные диски

- 6.1. Два вида дисков
- 6.2. Типы томов EBS
- 6.3. Том gp3: производительность отдельно от объёма
- 6.4. Том gp2: кредиты всплеска
- 6.5. Снимки и перенос данных
- 6.6. Что и за что тарифицируется

7. Модели оплаты вычислений

- 7.1. Планы экономии
- 7.2. Точка окупаемости обязательства
- 7.3. Прерываемые инстансы

8. Автоматическое масштабирование вычислительного слоя

- 8.1. Шаблон запуска и группа
- 8.2. Политики масштабирования

9. Выбор вычислительного сервиса

10. Ускорители вычислений и память моделей

- 10.1. Ускоренные инстансы AWS
- 10.2. Оценка памяти модели
- 10.3. Разобранный пример
- 10.4. Стоимость и модели оплаты

11. Вычислительные ресурсы для обучения и применения моделей

12. От локальной ВМ к облаку

- 12.1. Методика сравнительного измерения

13. Эталонная архитектура вычислительного слоя

14. Разобранные примеры

15. Задачи для самопроверки

16. Типичные ошибки и контрольные вопросы

17. Лабораторная работа № 4

- 17.1. Цель и порядок
- 17.2. Отчёт
- 17.3. Варианты
- 17.4. Задания
- 17.5. Задания повышенной сложности
- 17.6. Критерии
- 17.7. Этап сквозного проекта

18. Подготовка к итоговой аттестации

19. Приложения

- Приложение А. Чек-лист перед сдачей
- Приложение Б. Сводка команд для вычислительных ресурсов
- Приложение В. Глоссарий
- Приложение Г. Опорная библиотека: генератор котировок и индикаторы
- Приложение Д. Сквозной проект курса: состав, порядок ведения и защита

1. Цель занятия и требования к подготовке

Цель занятия

Сформировать понимание того, как одно физическое оборудование предоставляет изолированные вычислительные машины, каким образом эта технология лежит в основе облачного сервиса Amazon EC2 и как выбирается вычислительный ресурс под задачу, в том числе под задачу обучения и применения моделей машинного обучения на ускорителях.

Задачи занятия

1. Изучить формальные требования к виртуализации и причины, по которым архитектура x86 потребовала аппаратной поддержки.
2. Разобрать устройство гипервизоров первого и второго типов и режимы сети виртуальной машины.
3. Изучить систему AWS Nitro, образы машин, типы инстансов и их жизненный цикл.
4. Научиться оценивать объём памяти ускорителей, необходимый для инференса и обучения модели, и выбирать инстанс по документированным характеристикам.
5. Практически настроить виртуальную машину, измерить накладные расходы виртуализации и выполнить операции с инстансами через интерфейс командной строки AWS.

Требования к предварительной подготовке

— сборка проекта CMake в конфигурации Release (занятия 1 и 2);

— работа с интерфейсом командной строки AWS и эмулятором moto (занятие 3);

— понятия процесса, потока, виртуальной памяти из курса операционных систем;

— виртуальная машина Ubuntu, использованная на занятиях по CMake, либо возможность создать новую.

Ожидаемые результаты

— объяснять, почему для виртуализации x86 понадобились VT-x и AMD-V, и что такое VM exit;

— относить гипервизор к первому или второму типу и обосновывать отнесение;

— настраивать сеть ВМ под задачу: доступ извне, изоляция, связь между ВМ;

— описывать компоненты системы Nitro и объяснять, почему накладные расходы в EC2 малы;

— рассчитывать память ускорителей для модели заданного размера и выбирать инстанс;

— корректно проводить сравнительный замер производительности.

Вклад в сквозной проект. Занятие открывает работу над проектом: закрепляется паспорт варианта, измеряются накладные расходы виртуализации, выбираются тип инстанса и тип тома под расчётную нагрузку. Сохраните: файл схемы .drawio «регион — VPC — EC2» с изображением и таблицу расчётов по варианту. Опирается на занятия 1—3. Понадобится на занятиях 5 и 15: выбор ресурсов войдёт в модель стоимости проекта.

2. Рабочая среда

2.1. Проверка аппаратной поддержки виртуализации

Аппаратная поддержка виртуализации должна быть включена в прошивке (BIOS или UEFI) компьютера. Без неё современные гипервизоры не запустят 64-разрядную гостевую систему. Проверка выполняется средствами основной ОС.

ОС	Команда	Признак поддержки
Windows	`systeminfo` (раздел «Требования Hyper-V») или Диспетчер задач → Производительность → ЦП	«Виртуализация: включено»
Linux	`grep -c -E 'vmx|svm' /proc/cpuinfo` и `lscpu | grep -i virtualization`	число больше нуля; строка VT-x или AMD-V
macOS	`sysctl kern.hv_support`	значение 1

Обратите внимание. Флаги vmx (Intel) и svm (AMD) присутствуют только на процессорах архитектуры x86. На компьютерах Apple с процессорами серии M проверка выполняется командой `sysctl`; аппаратная поддержка виртуализации в них реализована средствами архитектуры ARM.

2.2. VirtualBox и образ Ubuntu

Используется Oracle VirtualBox версии 7.2. Если виртуальная машина Ubuntu осталась после занятий по CMake, её можно использовать повторно. Иначе создаётся новая ВМ с образом Ubuntu Server 24.04 LTS той же архитектуры, что и процессор хоста: x86-64 (amd64) для компьютеров Intel и AMD, ARM64 для компьютеров Apple на процессорах серии M. VirtualBox поддерживает хосты macOS на Apple Silicon начиная с версии 7.1 и запускает на них гостевые системы ARM64.

Важно. Одновременная работа VirtualBox и Hyper-V в Windows возможна, но снижает производительность: при включённом Hyper-V (он нужен для WSL 2 и Docker Desktop) VirtualBox выполняется поверх Windows Hypervisor Platform. На измерения это влияет; факт включения Hyper-V указывается в отчёте.

2.3. Эмулятор AWS

Используется среда из занятия 3: виртуальное окружение Python с пакетами `awscli` и `moto`, эмулятор запускается командой `moto_server -p 5001`. Если среда удалена, её следует восстановить по разделу 2 материалов занятия 3.

2.4. draw.io

Архитектурные схемы строятся в draw.io: веб-версия app.diagrams.net работает без регистрации, настольная версия устанавливается с сайта drawio.com. Для иконок AWS в левой панели выбирается «Ещё фигуры» (More Shapes) и включается библиотека AWS 2025 (в веб-адресе — параметр `libs=aws4`). Схема сохраняется в формате .drawio и экспортируется в PNG.

3. Виртуализация: теоретические основы

3.1. Понятие виртуальной машины

Виртуальная машина — эффективная изолированная копия вычислительной системы, выполняемая под управлением **монитора виртуальных машин**, или **гипервизора**. Гостевая операционная система в ВМ «видит» процессор, память и устройства так, будто она единственный владелец оборудования, хотя на самом деле ресурсы разделяются между несколькими ВМ.

Облачные вычисления стали возможны именно благодаря виртуализации: поставщик разделяет большие серверы на инстансы нужного размера, выделяет их за секунды и освобождает после использования. Характеристика «объединение ресурсов в пул», изученная на занятии 3, технически реализуется гипервизором.

3.2. Требования Попека — Голдберга

Формальные условия виртуализируемости архитектуры сформулированы Дж. Попеком и Р. Голдбергом в 1974 году. Монитор виртуальных машин должен обладать тремя свойствами.

1. **Эквивалентность.** Программа в ВМ ведёт себя так же, как на реальной машине, за исключением времени выполнения и доступного объёма ресурсов.
2. **Управление ресурсами.** Монитор полностью контролирует виртуализированные ресурсы: гость не может получить ресурсы, не выделенные ему явно.
3. **Эффективность.** Статистически преобладающая часть инструкций выполняется процессором непосредственно, без вмешательства монитора.

Для классической схемы «перехват и эмуляция» инструкции делятся на **привилегированные** — вызывающие исключение при выполнении вне режима ядра — и **чувствительные** — изменяющие или раскрывающие состояние, влияющее на распределение ресурсов. Основная теорема работы: архитектура виртуализируема методом перехвата, если множество чувствительных инструкций является подмножеством привилегированных. Тогда гостевое ядро можно запустить в непривилегированном режиме, и каждая опасная инструкция вызовет исключение, которое обработает монитор.



Рисунок 3.1 — Перехват чувствительных инструкций: программный и аппаратный подход

3.3. Проблема архитектуры x86 и способы её решения

Классическая архитектура x86 условию теоремы не удовлетворяла: ряд чувствительных инструкций (например, POPF, SGDT, SIDT) выполнялся в пользовательском режиме без исключения. Гостевое ядро, запущенное без привилегий, получало бы неверные сведения о состоянии процессора, а монитор об этом не узнавал бы. Решения появились в три этапа.

Подход	Суть	Цена
Двоичная трансляция	монитор просматривает код гостевого ядра и заменяет опасные фрагменты безопасными вызовами перед выполнением	сложность монитора, накладные расходы
Паравиртуализация	ядро гостя изменено и вместо чувствительных инструкций явно вызывает гипервизор (гипервызовы)	требуется модификация гостевой ОС
Аппаратная поддержка	Intel VT-x и AMD-V: отдельный режим non-root для гостя; события, заданные гипервизором, вызывают аппаратный выход в режим root	нужен процессор с поддержкой и её включение в прошивке

При аппаратной поддержке гостевое ядро выполняется на своём обычном уровне привилегий, но в режиме non-root. Когда происходит событие, которое гипервизор пожелал перехватывать (чувствительная инструкция, внешнее прерывание, обращение к устройству), процессор выполняет **выход из ВМ (VM exit)**: сохраняет состояние гостя в управляющую структуру (VMCS у Intel, VMCB у AMD) и передаёт управление гипервизору. После обработки выполняется **вход в ВМ (VM entry)**.

> **Обратите внимание.** Каждый выход из ВМ стоит сотни и тысячи тактов процессора. Поэтому производительность виртуализации определяется прежде всего числом выходов: вычислительная задача без ввода-вывода почти их не вызывает и работает со скоростью, близкой к нативной, а интенсивный ввод-вывод через эмулируемые устройства вызывает их постоянно.

3.4. Виртуализация памяти и ввода-вывода

Память. Гостевая ОС управляет «физическими» адресами, которые на самом деле являются адресами внутри памяти, выделенной ВМ. Нужно двухуровневое преобразование: гостевой виртуальный адрес → гостевой физический → физический адрес хоста. Без аппаратной помощи гипервизор ведёт **теневые таблицы страниц** и перехватывает каждое их изменение. Трансляция второго уровня — **EPT** у Intel и **NPT (RVI)** у AMD — выполняет второе преобразование аппаратно и устраняет эти перехваты.

Ввод-вывод. Устройства виртуализируются тремя способами:

— **эмуляция** реального устройства (например, распространённой сетевой карты) — работает с любой ОС, но каждое обращение к регистрам устройства вызывает выход из ВМ;

— **паравиртуальные драйверы** (virtio в KVM, Guest Additions в VirtualBox) — гость знает, что работает в ВМ, и обменивается с гипервизором очередями запросов, что резко сокращает число выходов;

— **проброс устройства** (SR-IOV, PCI passthrough) — ВМ получает прямой доступ к физическому устройству или его виртуальной функции; так подключаются GPU и высокоскоростные сетевые адаптеры.

3.5. Виртуальная машина и контейнер: предварительное сравнение

Виртуальная машина виртуализирует оборудование: у каждой ВМ собственное ядро ОС. Контейнер, изучаемый на следующем занятии, виртуализирует окружение процесса: все

контейнеры на узле используют общее ядро основной ОС, а изоляция обеспечивается механизмами самого ядра. Отсюда основные различия, которые определяют выбор между ними.

Свойство	Виртуальная машина	Контейнер
Что виртуализируется	оборудование	окружение процесса в общем ядре
Гостевая ОС	полная, со своим ядром	только пользовательская часть
Время запуска	десятки секунд — минуты	доли секунды — секунды
Размер образа	гигабайты	мегабайты — сотни мегабайт
Изоляция	сильная: граница — гипервизор	слабее: граница — ядро ОС
Другая ОС (Windows в Linux)	возможна	невозможна: ядро общее

В облаке оба подхода сочетаются: контейнеры запускаются внутри виртуальных машин (инстансов) либо в микро-ВМ, как в AWS Fargate и AWS Lambda, где для каждого клиента создаётся облегчённая виртуальная машина Firecracker. Так достигается скорость контейнеров при изоляции виртуальных машин.

4. Гипервизоры

4.1. Первый и второй типы

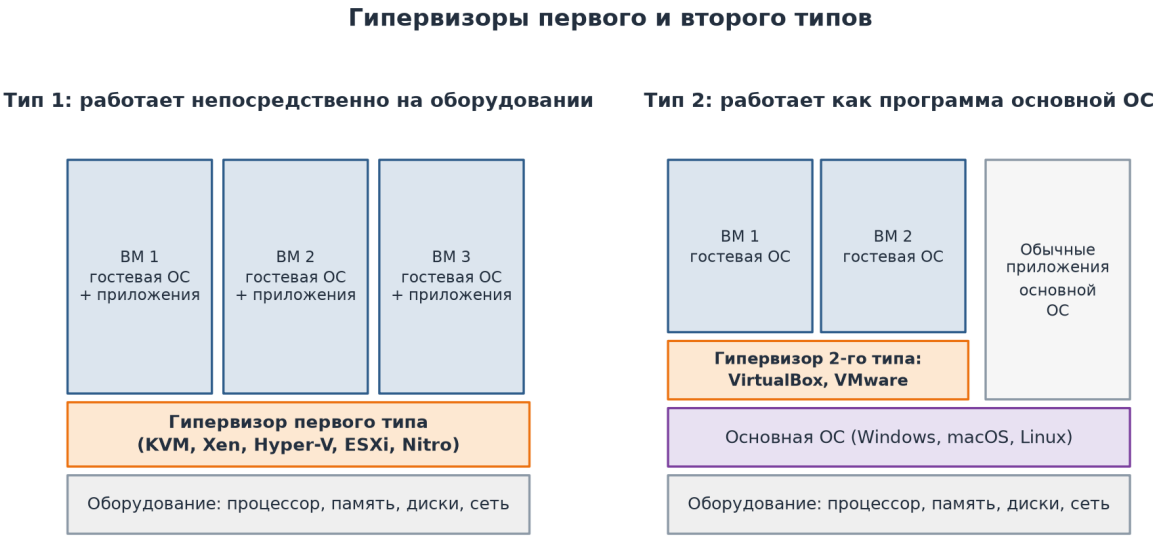


Рисунок 4.1 — Гипервизоры первого и второго типов

Гипервизор первого типа работает непосредственно на оборудовании и сам управляет процессором и памятью: VMware ESXi, Microsoft Hyper-V, Xen, KVM, гипервизор Nitro. Такие гипервизоры применяются в центрах обработки данных и облаках. **Гипервизор второго типа** работает как приложение основной ОС и пользуется её драйверами и планировщиком: Oracle VirtualBox, VMware Workstation, Parallels Desktop. Он удобен на рабочем месте разработчика.

4.2. KVM, VirtualBox, VMware

KVM (Kernel-based Virtual Machine) — модуль ядра Linux, использующий VT-x или AMD-V. Он превращает ядро Linux в гипервизор: каждая ВМ является обычным процессом, а её виртуальные процессоры — потоками, которые планирует ядро. Эмуляцию устройств обычно выполняет QEMU. KVM относят к первому типу, так как гипервизором является само ядро, работающее на оборудовании; при этом ядро остаётся ОС общего назначения, поэтому классификация условна.

VirtualBox — гипервизор второго типа с открытым исходным кодом (основная часть — GPL). Управляется графически и из командной строки утилитой `VBoxManage`. **VMware Workstation** — аналогичный продукт второго типа; серверный **VMware ESXi** относится к первому типу.

Основные команды VBoxManage (ВМ с именем ubuntu)

```
VBoxManage list vms                                  # список ВМ
VBoxManage modifyvm "ubuntu" --cpus 2 --memory 3072   # число vCPU и память (МБ)
VBoxManage modifyvm "ubuntu" --natpf1 "ssh,tcp,,2201,,22"   # проброс хост:2201 -> ВМ:22
VBoxManage modifyvm "ubuntu" --nic2 hostonly --hostonlyadapter2 "vboxnet0"
VBoxManage startvm "ubuntu" --type headless          # запуск без окна
VBoxManage snapshot "ubuntu" take "clean"            # снимок
VBoxManage snapshot "ubuntu" restore "clean"         # возврат (ВМ должна быть
остановлена)
```

Обратите внимание. Изменять конфигурацию командой `modifyvm` можно только у выключенной ВМ. Имя адаптера хоста (`vboxnet0` в Linux и macOS) в Windows выглядит иначе, например «VirtualBox Host-Only Ethernet Adapter»; точное имя выводит команда `VBoxManage list hostonlyifs`.

4.3. Сеть виртуальной машины



Рисунок 4.2 — Режимы сетевого адаптера VirtualBox

Для учебной задачи удобно сочетание двух адаптеров: **NAT** даёт ВМ выход в Интернет для установки пакетов, **виртуальный адаптер хоста** — постоянный адрес, по которому хост обращается к ВМ. Этот же приём используется в облаке: инстанс в частной подсети выходит наружу через NAT-шлюз, а служебный доступ организуется по внутреннему адресу.

5. Вычислительные ресурсы облака: Amazon EC2

5.1. Система Nitro

Современные инстансы EC2 работают на **системе AWS Nitro**. Её идея — вынести функции виртуализации из программного гипервизора в специализированное оборудование.

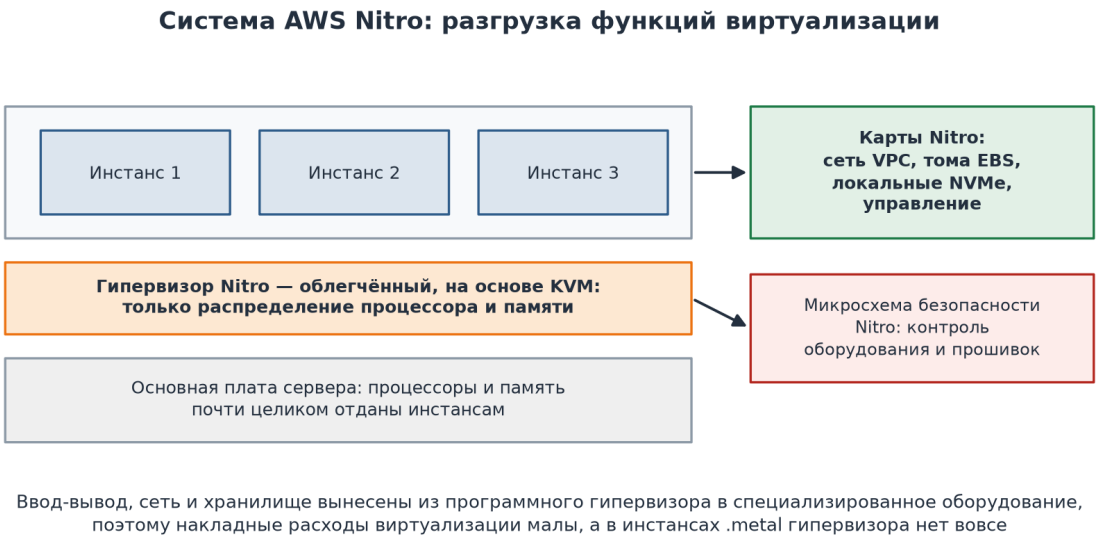


Рисунок 5.1 — Компоненты системы Nitro

— **карты Nitro** реализуют сеть VPC, тома EBS, локальные диски NVMe и управление сервером;

— **микросхема безопасности Nitro** контролирует оборудование и прошивки сервера;

— **гипервизор Nitro** — облегчённый гипервизор на основе KVM; за ним остаётся только распределение процессорного времени и памяти между инстансами.

Поскольку ввод-вывод обслуживают карты, число выходов из ВМ при сетевом и дисковом обмене невелико, и производительность инстанса близка к производительности физического сервера. Инстансы с суффиксом `.metal` (например, `c5.metal`) предоставляют сервер целиком, без гипервизора; сеть и тома EBS при этом по-прежнему обеспечивают карты Nitro.

5.2. Образы, инстансы, жизненный цикл

Инстанс запускается из **образа машины (AMI)**, содержащего ОС и программное обеспечение. Тип инстанса определяет ресурсы (схема именования разобрана на занятии 3: серия, поколение, признаки, размер). Жизненный цикл инстанса:

Состояние	Смысл	Оплата вычислений
pending	инстанс готовится к запуску	нет
running	инстанс работает	да
stopping / stopped	остановка / инстанс остановлен, тома EBS сохранены	нет (тома — да)
shutting-down / terminated	удаление / инстанс удалён безвозвратно	нет

5.3. Семейства инстансов и понятие vCPU

Типы инстансов сгруппированы по назначению. Выбор семейства определяется отношением памяти к процессору и наличием специализированного оборудования.

Семейство	Серии	Память на vCPU	Типичная нагрузка
Универсальные	M, T	4 ГиБ (M), переменная производительность (T)	веб-приложения, микросервисы
Вычислительные	C	2 ГиБ	расчёты, пакетная обработка, кодирование видео
С увеличенной памятью	R, X	8 ГиБ (R) и больше	базы данных, кэши, анализ в памяти
С ускорителями	P, G, Trn, Inf	определяется ускорителями	обучение и инференс моделей, графика
С быстрым хранилищем	I, D	зависит от серии	распределённые файловые системы, NoSQL

Отношение памяти к vCPU проверяется на примерах из эмулятора и документации: m5.large — 2 vCPU и 8 ГиБ, c5.xlarge — 4 vCPU и 8 ГиБ. **vCPU** — единица процессорной мощности инстанса. В инстансах на процессорах Intel и AMD vCPU соответствует аппаратному потоку ядра (при одновременной многопоточности на одно ядро приходятся два vCPU), в инстансах на процессорах AWS Graviton — физическому ядру. Поэтому одинаковое число vCPU у инстансов разных архитектур не означает одинаковой производительности.

5.4. Хранилище инстанса

Инстанс использует два вида дисков. **Тома Amazon EBS** — сетевые блочные устройства, которые существуют независимо от инстанса: при остановке инстанса данные на них сохраняются, том можно отключить и подключить к другому инстансу. **Локальные диски NVMe** (хранилище инстанса, признак `d` в имени типа) физически расположены на сервере: они быстрее, но их содержимое теряется при остановке или удалении инстанса. Поэтому локальные диски используются для временных данных — кэша, промежуточных файлов обработки, копии обучающей выборки, — а результаты сохраняются в EBS или S3.

5.5. Пользовательские данные

При запуске инстанса можно передать **пользовательские данные (user data)** — сценарий оболочки или конфигурацию cloud-init. По умолчанию они выполняются один раз, при первой загрузке, с правами суперпользователя. Это аналог сценария, который администратор выполняет вручную после установки ОС в VirtualBox, но выполняемого автоматически и воспроизводимо.

Файл user-data.sh

```
#!/bin/bash
# Пользовательские данные: выполняются один раз при первом запуске инстанса (cloud-init)
set -e
apt-get update
apt-get install -y build-essential cmake
echo "Инстанс варианта NN подготовлен: $(date -u)" > /etc/motd
```

Запуск инстанса с пользовательскими данными и их чтение (адрес эмулятора опущен)

```
aws ec2 run-instances --image-id ami-12345678 --instance-type t3.small \
    --user-data file://user-data.sh \
    --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=var01}]' \
    --query "Instances[0].InstanceId" --output text

aws ec2 describe-instance-attribute --instance-id <ID> --attribute userData \
    --query "UserData.Value" --output text | base64 --decode
```

5.6. Регионы, зоны доступности и размещение

Физически облако состоит из **регионов** — географически разделённых площадок. Регион состоит из нескольких **зон доступности**; зона — это один или несколько центров обработки данных с независимым питанием, охлаждением и сетью. Зоны одного региона соединены отдельными высокоскоростными каналами с задержкой в единицы миллисекунд, а регионы между собой — обычной магистральной сетью с задержкой в десятки и сотни миллисекунд.

Уровень	Что это	Задержка	Что переживает
Регион	географическая площадка	десятки — сотни мс между регионами	отказ региона (при копии в другом)
Зона доступности	независимый центр обработки данных	единицы мс между зонами	отказ одной зоны
Группа размещения	правило соседства инстансов	доли мс внутри группы	зависит от вида группы

Из этой структуры следуют три правила проектирования, которые применяются в каждой лабораторной работе курса:

1. рабочие копии службы размещаются **не менее чем в двух зонах** — иначе отказ зоны останавливает обслуживание целиком;
2. данные, которые нельзя потерять, копируются между зонами (объектное хранилище делает это само) или между регионами (копированием снимков);
3. обмен между зонами оплачивается и добавляет задержку, поэтому «болтливые» части системы размещают в одной зоне, а избыточность обеспечивают на уровне службы, а не отдельного запроса.

Группы размещения задают, как инстансы располагаются относительно друг друга. Кластерная группа собирает инстансы в одной стойке ради наименьшей задержки — так размещают узлы распределённого обучения; распределённая группа, наоборот, разносит инстансы по разному оборудованию, чтобы отказ одной стойки не затронул более одного инстанса.

Обратите внимание. Номер зоны в имени (`us-east-1a`) у разных учётных записей соответствует разному физическому оборудованию: имена зон отображаются на физические зоны индивидуально. Поэтому «положить всё в зону a» не означает, что все окажутся рядом; для согласования используется идентификатор зоны.

5.7. Образы машин и воспроизводимость среды

Образ машины (AMI) — это снимок загрузочного тома вместе с описанием: архитектура, режим запуска, устройства. Образ решает ту же задачу, что и Dockerfile, но на уровне целой машины: он делает среду воспроизводимой. Различие в зерне: образ фиксирует **результат** установки, Dockerfile — **шаги** установки.

Признак	Образ машины	Dockerfile и образ контейнера
Что фиксируется	состояние диска целиком	последовательность шагов сборки
Размер	гигабайты	десятки — сотни мегабайт
Время запуска	десятки секунд	доли секунды
Что внутри	ядро, драйверы, системные службы	только пользовательское пространство
Как проверяется	запуском инстанса	повторной сборкой по тому же файлу
Где применяется	базовая среда узла	приложение и его зависимости

На практике два подхода сочетаются: образ машины содержит операционную систему, агенты сбора метрик и исполнитель контейнеров, а приложение поставляется образом контейнера. Так обновление приложения не требует пересборки образа машины, а обновление ядра не задевает приложение.

5.8. Ограничения учётной записи

Ресурсы облака не бесконечны для потребителя: на каждый сервис действуют **квоты** — предельное число инстансов, адресов, томов, правил. Квоты задаются на регион и на учётную запись, часть из них увеличивается по запросу, часть — жёсткая. Типичная ошибка при первом развёртывании — упереться в квоту на число vCPU выбранного семейства и получить отказ запуска в самый неудобный момент.

— квоты проверяются **до** развёртывания, а не после отказа;

— автоматическое масштабирование ограничивается не только параметром maximum, но и квотой;

— в учебных заданиях курса квоты не достигаются, но привычку проверять их следует выработать сразу.

6. Блочное хранилище: тома EBS и локальные диски

Вычислительный ресурс без хранилища бесполезен. В облаке диск инстанса — это не пластина внутри корпуса, а **сетевое блочное устройство**, которое существует отдельно от вычислительного узла. Понимание этого различия объясняет и цену, и производительность, и поведение при сбое.

6.1. Два вида дисков

Признак	Том Amazon EBS	Локальный диск инстанса
Расположение	сеть хранения, отдельно от сервера	физически на том же сервере
Жизненный цикл	переживает остановку и удаление инстанса	стирается при остановке и удалении
Перенос	отключается и подключается к другому инстансу	перенести нельзя
Снимки	поддерживаются, инкрементные	нет, копируют вручную
Задержка	единицы миллисекунд (io2 — доли миллисекунды)	десятки микросекунд
Оплата	за выделенный объём и заказанную производительность	включена в цену инстанса
Назначение	данные, состояние, загрузочный том	кэш, временные файлы, копия выборки

Локальный диск обозначается признаком `d` в имени типа инстанса: `m5d.large`, `c5d.xlarge`, `i4i.2xlarge`. Правило простое: **на локальном диске лежит то, что не жалко потерять**. Обучающая выборка, скопированная из объектного хранилища, промежуточные файлы сортировки, кэш разобранных данных — да; результаты расчёта, журнал операций, база данных — нет.

> **Важно.** Остановка инстанса (`stop`) и его перезагрузка (`reboot`) — разные события. Перезагрузка сохраняет содержимое локального диска, остановка — нет: при следующем запуске инстанс может оказаться на другом физическом сервере.

6.2. Типы томов EBS



Рисунок 6.1 — Предельные показатели типов томов

Тома делятся на два класса: на твердотельных накопителях (оптимизированы под число операций) и на жёстких дисках (оптимизированы под поток). Характеристики приведены по документации Amazon EBS.

Тип	Размер	Операций в секунду	Пропускная способность	Долговечность в год
gp3	1 ГиБ — 64 ТиБ	до 80 000	до 2000 МиБ/с	99,8—99,9 %
gp2	1 ГиБ — 16 ТиБ	до 16 000	128—250 МиБ/с	99,8—99,9 %
io2 Block Express	4 ГиБ — 64 ТиБ	до 256 000	до 4000 МиБ/с	99,999 %
io1	4 ГиБ — 16 ТиБ	до 64 000	до 1000 МиБ/с	99,8—99,9 %
st1 (жёсткие диски)	125 ГиБ — 16 ТиБ	500 (блок 1 МиБ)	до 500 МиБ/с	99,8—99,9 %
sc1 (жёсткие диски)	125 ГиБ — 16 ТиБ	250 (блок 1 МиБ)	до 250 МиБ/с	99,8—99,9 %

Долговечность 99,8—99,9 % в год означает ожидаемые не более двух отказов на тысячу работающих томов за год. У io2 Block Express заявлена долговечность 99,999 % — отказ одного тома из ста тысяч за год; этот тип рассчитан на базы данных, где потеря тома недопустима. Предельные 256 000 операций достижимы только на инстансах системы Nitro; остальные типы инстансов ограничены 32 000 операций.

Обратите внимание. Тома `st1` и `sc1` не могут быть загрузочными и оперируют блоками по 1 МиБ: 500 операций в секунду означают именно 500 мегабайтных блоков. На случайном мелком чтении такой том работает медленно — он рассчитан на последовательный поток, то есть на журналы, архивы и хранилища данных.

6.3. Том gp3: производительность отдельно от объёма

Главное отличие gp3 от предыдущего поколения — **производительность настраивается независимо от размера**. В цену тома включены 3000 операций в секунду и 125 МиБ/с; их достаточно для загрузочного тома и умеренной нагрузки. Дополнительная производительность заказывается и оплачивается отдельно, в пределах двух соотношений:

— не более **500 операций в секунду на гибибайт** объёма: предельные 80 000 операций доступны на томе от 160 ГиБ (500 × 160 = 80 000);

— не более **0,25 МиБ/с на одну заказанную операцию**: предельные 2000 МиБ/с доступны при 8000 операций и размере тома не менее 16 ГиБ.

Ограничения объясняют типичную ошибку проектирования. Требование «том на 20 ГиБ с 20 000 операций в секунду» невыполнимо: при 500 операциях на гибибайт для 20 000 операций нужен том не менее 40 ГиБ. Требование «500 МиБ/с при 1000 операций» тоже невыполнимо: 0,25 × 1000 = 250 МиБ/с.

Разобранный пример. Сервису нужно 6000 операций в секунду и 400 МиБ/с. Проверяем: минимальный размер тома 6000 / 500 = 12 ГиБ, но для 2000 МиБ/с потребовалось бы 8000 операций; нам нужно 400 МиБ/с, значит достаточно 400 / 0,25 = 1600 операций. Ограничение по операциям сильнее: заказываем 6000 операций и 400 МиБ/с на томе от 16 ГиБ. Сверх включённых значений оплачиваются 3000 операций и 275 МиБ/с.

Связь с практикой. Тома gp3 не используют всплески: заказанная производительность доступна постоянно. Это делает измерения воспроизводимыми — что особенно важно, когда по результатам измерения принимается решение о выборе ресурса.

6.4. Том gp2: кредиты всплеска

В томах предыдущего поколения gp2 производительность жёстко связана с объёмом: **3 операции в секунду на гибибайт**, но не менее 100 (тома до 33,33 ГиБ) и не более 16 000 (достигается на томе 5334 ГиБ). Небольшой том с базовой производительностью 100—300 операций не справился бы с запуском системы, поэтому введён механизм кредитов.

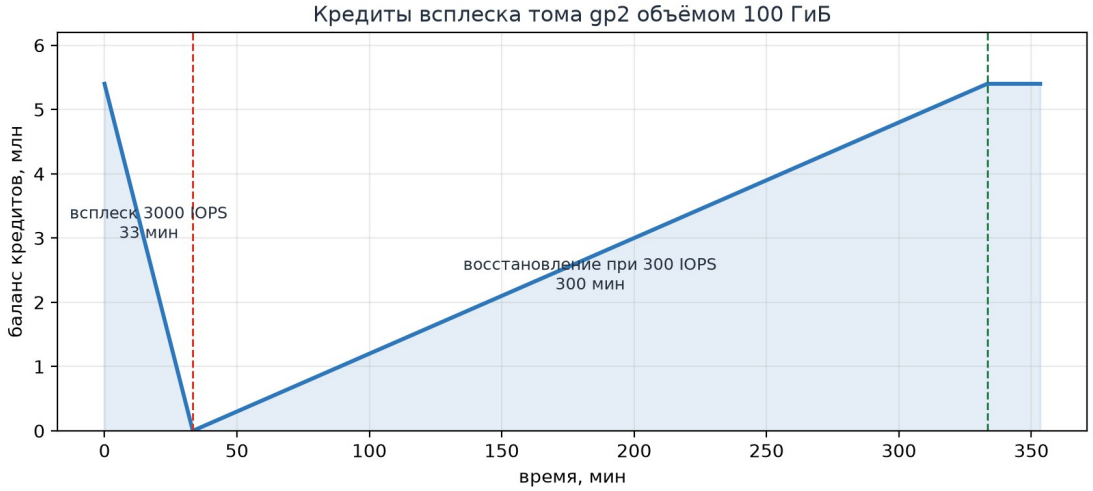


Рисунок 6.2 — Баланс кредитов всплеска тома gp2 объёмом 100 ГиБ

— том получает начальный баланс **5,4 млн кредитов** и накапливает 3 кредита на гибибайт в секунду;

— предел накопления — те же 5,4 млн кредитов;

— при нагрузке выше базовой том расходует кредиты и работает со скоростью до **3000 операций** в секунду;

— расход в секунду равен превышению над базовой производительностью.

Отсюда формула длительности всплеска: **время = баланс ÷ (всплеск − база)**. Для тома 100 ГиБ база равна 300 операций, расход при полном всплеске 3000 − 300 = 2700 кредитов в секунду, длительность 5 400 000 / 2700 = 2000 секунд, то есть около 33 минут. Восстановление пустого баланса занимает 5 400 000 / 300 = 18 000 секунд — ровно 5 часов.

Размер тома, ГиБ	База, операций/с	Всплеск при 3000 операций/с	Восстановление баланса
1—33,33	100	1862 с ≈ 31 мин	54 000 с = 15 ч
100	300	2000 с ≈ 33 мин	18 000 с = 5 ч
334	1002	2703 с ≈ 45 мин	5389 с ≈ 1,5 ч
750	2250	7200 с = 2 ч	2400 с = 40 мин
1000 и больше	3000 и выше	всплеск не нужен	—

Важно. Отсюда происходит одна из самых неприятных ошибок эксплуатации: система месяцами работает нормально, а под нагрузкой внезапно «проседает» и не восстанавливается. Причина — исчерпанный баланс кредитов: метрика `BurstBalance` в CloudWatch показывает его в процентах, и сигнал нужно настраивать заранее.

6.5. Снимки и перенос данных

Снимок (snapshot) — копия тома в объектном хранилище. Снимки инкрементные: первый сохраняет все занятые блоки, последующие — только изменившиеся с прошлого раза. Удаление промежуточного снимка не ломает остальные: блоки, нужные более поздним снимкам,

сохраняются. Из снимка создаётся новый том, в том числе в другой зоне доступности или, после копирования снимка, в другом регионе — это штатный способ переноса данных и восстановления после сбоя зоны.

Снимки участвуют в трёх сценариях: резервное копирование по расписанию, создание одинаковых рабочих сред (снимок → образ машины → инстансы), перенос между регионами. Шифрование тома наследуется снимками, а снимок зашифрованного тома нельзя сделать открытым.

6.6. Что и за что тарифицируется

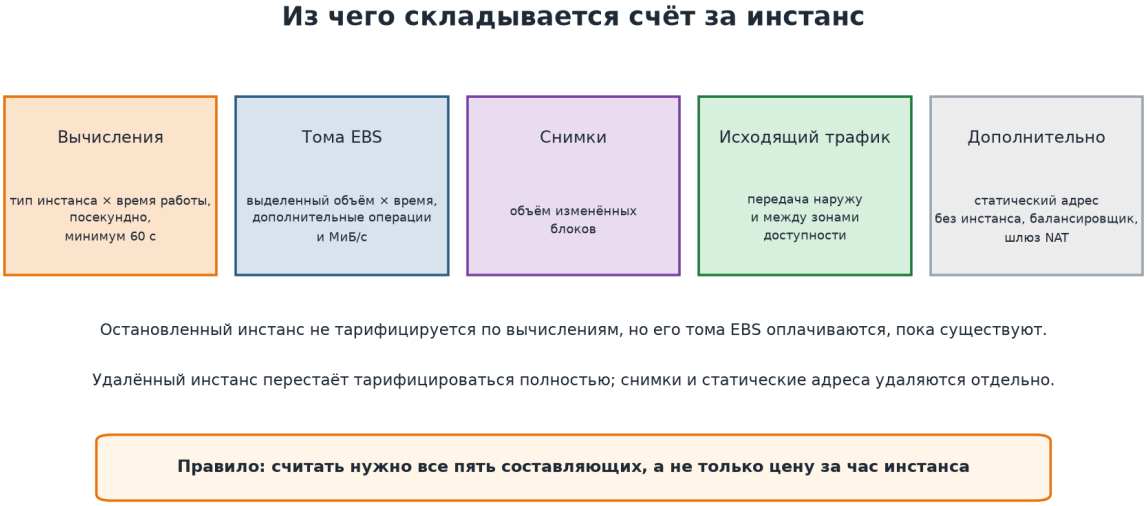


Рисунок 6.3 — Составляющие счёта за вычислительный ресурс

Счёт за вычислительный ресурс складывается не из одной ставки. Вычисления тарифицируются посекундно с минимумом в одну минуту (для Linux) и только в состоянии `running`. Тома EBS оплачиваются за **выделенный**, а не за занятый объём, всё время существования тома, даже если инстанс остановлен. Снимки оплачиваются за объём изменённых блоков. Отдельно тарифицируется исходящий трафик и постоянный сетевой адрес, не привязанный к работающему инстансу.

Составляющая	Когда начисляется	Как уменьшить
Вычисления	инстанс в состоянии running	остановка вне работы, планы экономии, прерываемые инстансы
Том EBS	всё время существования тома	удалять тома вместе с инстансом, не завышать объём
Заказанные операции и МиБ/с	сверх включённых в gp3	проверять реальную потребность по метрикам
Снимки	объём изменённых блоков	политика хранения, удаление устаревших снимков
Исходящий трафик	передача за пределы региона	кэширование, сжатие, обработка ближе к данным
Статический адрес	адрес выделен, но не используется	освобождать адреса вместе с инстансом

Обратите внимание. Учебные расчёты в задании выполняются по ставкам из таблицы вариантов. Действующие ставки региона всегда сверяются на странице тарифов сервиса: цены меняются, и решение, принятое по устаревшей ставке, оказывается неверным.

7. Модели оплаты вычислений

Один и тот же инстанс можно оплачивать пятью способами. Выбор способа меняет стоимость в разы и относится к проектным решениям, а не к бухгалтерии: он определяет, можно ли остановить работу в любой момент и насколько жёстко система привязана к конкретному семейству инстансов.

Способ оплаты	Обязательство	Когда применяется	Риск
По требованию	нет	неизвестная или редкая нагрузка, эксперименты	самая высокая цена
План экономии	сумма в час на 1 или 3 года	постоянная базовая нагрузка	обязательство платить
Зарезервированные инстансы	конкретный тип на 1 или 3 года	устоявшаяся конфигурация	привязка к типу
Прерываемые (Spot)	нет	пакетные расчёты, обучение с контрольными точками	изъятие за 2 минуты
Выделенный сервер	оплата сервера целиком	требования лицензий и изоляции	низкая утилизация

7.1. Планы экономии

План экономии — обязательство тратить не менее заданной суммы в час в течение года или трёх лет; в обмен на обязательство действует пониженная ставка. Планов четыре типа, и они различаются не размером скидки, а тем, на что распространяются.

Тип плана	На что распространяется	Гибкость
План для вычислений	EC2, Fargate, Lambda в любом регионе и семействе	наибольшая
План для инстансов EC2	выбранное семейство в выбранном регионе	смена размера внутри семейства
План для SageMaker	вычисления машинного обучения	внутри сервиса
План для баз данных	управляемые базы данных	внутри сервиса

Скидка тем больше, чем уже обязательство: план для инстансов EC2 даёт больше, чем план для вычислений, но лишает свободы сменить семейство. Верхние границы заявленной экономии относительно оплаты по требованию: до 66 % для плана вычислений, до 72 % для плана инстансов EC2, до 35 % для плана баз данных. Это верхние границы: фактическая скидка зависит от срока, способа оплаты и профиля нагрузки.

7.2. Точка окупаемости обязательства

План экономии выгоден не всегда. Обязательство оплачивается круглосуточно, а нагрузка может существовать не всё время. Пусть s — доля скидки (например, 0,66), тогда план окупается, если доля времени работы ресурса превышает $\mathbf{1 - s}$.

Вывод формулы прямой. Пусть цена по требованию равна C за час, доля времени работы равна k. Оплата по требованию за месяц: $C \cdot k \cdot 730$. Оплата по плану: $C \cdot (1 - s) \cdot 730$ — она не зависит от k, потому что обязательство действует всегда. План выгоден при $C \cdot k \cdot 730 > C \cdot (1 - s) \cdot 730$, то есть при $k > 1 - s$.

Скидка плана	Порог по времени работы	Смысл
66 %	34 % времени	примерно 8 часов в сутки пять дней в неделю
72 %	28 % времени	примерно 7 часов в сутки пять дней в неделю
35 %	65 % времени	нагрузка почти круглосуточная

Отсюда практическое правило: круглосуточные службы переводят на план экономии, а среды разработки, работающие в рабочее время, дешевле останавливать на ночь и оплачивать по требованию. Экономия от остановки среды разработки на ночь и выходные обычно больше, чем от любого плана.

7.3. Прерываемые инстансы

Прерываемый инстанс (Spot) использует временно свободную ёмкость и стоит существенно дешевле оплаты по требованию — заявленная экономия достигает 90 %. Плата за это — право поставщика забрать ёмкость: инстанс получает уведомление о прерывании за две минуты и затем останавливается или удаляется. Дополнительно публикуется рекомендация к перебалансировке — сигнал о повышенном риске изъятия, приходящий раньше самого уведомления.

Отсюда требования к программе, которая работает на прерываемых инстансах:

1. работа делится на части, каждая из которых завершается за минуты;
2. состояние регулярно сохраняется в контрольной точке во внешнем хранилище;
3. обработчик уведомления успевает сохранить состояние и корректно завершиться;
4. очередь заданий возвращает незавершённое задание другому исполнителю;
5. результат не зависит от того, сколько раз задание прерывалось и перезапускалось.

Связь с практикой. Обучение модели — типичный кандидат на прерываемые инстансы: контрольные точки сохраняются каждые несколько минут, и после прерывания обучение продолжается с последней точки. Промышленный сервис, обслуживающий запросы пользователей, на прерываемых инстансах не размещают — либо размещают только избыточную часть реплик.

Важно. Прерываемая ёмкость не гарантируется: при нехватке свободной ёмкости запрос просто не выполняется. Поэтому в описании группы масштабирования обычно смешивают оплату по требованию (базовая часть реплик) и прерываемые инстансы (часть сверх базовой).

8. Автоматическое масштабирование вычислительного слоя

Вместо того чтобы держать инстансы «с запасом на пик», в облаке описывают **желаемое состояние**: сколько реплик должно работать и по какому признаку это число меняется. Инфраструктура сама запускает и останавливает инстансы.



Рисунок 8.1 — Группа автоматического масштабирования

8.1. Шаблон запуска и группа

Шаблон запуска фиксирует всё, что нужно для создания одинаковых инстансов: образ машины, тип инстанса, пользовательские данные, роль, группы безопасности, параметры томов. **Группа масштабирования** задаёт минимальное, желаемое и максимальное число инстансов и список зон доступности. Группа поддерживает желаемое число работающих и **исправных** инстансов: инстанс, не прошедший проверку состояния, удаляется и заменяется новым.

Параметр группы	Смысл	Типичная ошибка
minimum	нижняя граница числа инстансов	ноль у службы, которая должна отвечать всегда
desired	текущее желаемое число	правка вручную вместо изменения политики
maximum	верхняя граница	слишком низкая: рост нагрузки упирается в предел
зоны доступности	где размещаются инстансы	одна зона: отказ зоны останавливает службу
проверка состояния	EC2 или проверка балансировщика	проверка «процесс запущен» вместо «отвечает»
время прогрева	сколько новый инстанс не учитывается в метрике	ноль: система масштабируется рывками

8.2. Политики масштабирования

— целевое отслеживание — поддерживать метрику около заданного значения (например, среднюю загрузку процессора 55 %); служба сама увеличивает и уменьшает число инстансов;

— **пошаговое** — заданы пороги и величина шага: при превышении первого порога добавить один инстанс, второго — три;

— **по расписанию** — число инстансов меняется в известные моменты: перед началом торговой сессии, перед ночным расчётом отчётов.

Оценить требуемое число инстансов до настройки политики можно пропорцией: нужно = ⌈текущее × текущая метрика ÷ целевое значение⌉. Например, при трёх инстансах, средней

загрузке 110 % от цели и цели 55 % получаем ⌈3 × 110 / 55⌉ = 6 инстансов. Это оценка: служба меняет ёмкость постепенно и учитывает время прогрева, чтобы не раскачивать систему.

Масштабирование имеет пределы, о которых нужно помнить. Новый инстанс появляется не мгновенно: запуск, загрузка образа, прогрев кэшей занимают от десятков секунд до минут. Поэтому автоматическое масштабирование не спасает от мгновенного всплеска — от него защищают очередь, ограничение частоты запросов и запас ёмкости.

Важно. Масштабирование вычислительного слоя бесполезно, если узкое место в другом месте: в базе данных, во внешнем сервисе или в блокировке. Увеличение числа инстансов в этом случае лишь увеличивает счёт и нагрузку на узкое место.

9. Выбор вычислительного сервиса

Инстанс — не единственный способ выполнить код в облаке. Для одной и той же задачи обычно есть три-четыре варианта, и они различаются тем, чем именно управляет потребитель.



Рисунок 9.1 — Выбор вычислительного сервиса

Признак	Amazon EC2	AWS Fargate	AWS Lambda
Единица развёртывания	виртуальная машина	задача из контейнеров	функция
Чем управляет потребитель	ОС, драйверы, обновления	образ и параметры задачи	только код и память
Тарификация	посекундно, минимум 60 с	посекундно, минимум 60 с	за время работы и память
Предельное время работы	не ограничено	не ограничено	900 секунд
Масштабирование	группа масштабирования	число задач службы	автоматически, по событиям
Холодный старт	минуты (запуск инстанса)	десятки секунд (запуск задачи)	сотни миллисекунд
Планы экономии и Spot	да, включая прерываемые	план для вычислений, Spot	план для вычислений
Когда выбирают	нужен контроль над узлом	служба в контейнере без узлов	короткая обработка события

Разобранный пример. Служба принимает 20 запросов в секунду, каждый обрабатывается 40 мс. Средняя занятость — 20 × 0,04 = 0,8 одновременного обработчика, то есть нагрузка мала, но постоянна. Для функции это 1,73 млн вызовов в месяц — тарификация за время работы окажется заметной, а для задачи контейнера достаточно половины vCPU. При постоянной нагрузке контейнерная задача дешевле функции; при редких обращениях — наоборот, потому что простаивающая задача оплачивается, а незапущенная функция — нет.

Отдельный случай — пакетные расчёты. Сервис пакетной обработки принимает очередь заданий и сам подбирает вычислительные ресурсы, в том числе прерываемые. Это естественный способ считать сценарии Монте-Карло, перекодировать видео или обучать модель на наборе конфигураций.

Связь с практикой. Правило выбора: сначала определяется характер нагрузки (постоянная, редкая, пакетная) и нужный уровень контроля над средой, и только потом сравнивается стоимость. Обратный порядок приводит к решениям, которые дешевле на бумаге и дороже в эксплуатации.

10. Ускорители вычислений и память моделей

10.1. Ускоренные инстансы AWS

Для обучения и применения моделей используются инстансы с ускорителями: графическими процессорами NVIDIA и собственными чипами AWS. Характеристики в таблице взяты из документации AWS по типам ускоренных инстансов (сентябрь 2026 года).

Таблица 10.1 — Ускоренные инстансы (источник: документация AWS)

Инстанс	Ускорители	Память, ГиБ	vCPU	ОЗУ, ГиБ	Назначение
g4dn.xlarge	1 × NVIDIA T4	16	4	16	недорогой инференс
g5.xlarge	1 × NVIDIA A10G	22	4	16	инференс, дообучение небольших моделей
g6e.xlarge	1 × NVIDIA L40S	44	4	32	инференс моделей среднего размера
g5.12xlarge	4 × NVIDIA A10G	89	48	192	дообучение, инференс с разбиением
p4d.24xlarge	8 × NVIDIA A100	320 (8 × 40)	96	1152	обучение
p4de.24xlarge	8 × NVIDIA A100	640 (8 × 80)	96	1152	обучение крупных моделей
p5.48xlarge	8 × NVIDIA H100	640 (8 × 80)	192	2048	обучение крупных моделей
trn1.32xlarge	16 × AWS Trainium	512 (16 × 32)	128	512	обучение на чипах AWS
inf2.48xlarge	12 × AWS Inferentia2	384 (12 × 32)	192	768	инференс на чипах AWS

Обратите внимание. Инстансы p4d.24xlarge и p4de.24xlarge отличаются именно памятью ускорителей: 40 и 80 ГиБ на каждый A100. Это различие существенно при выборе инстанса для крупных моделей.

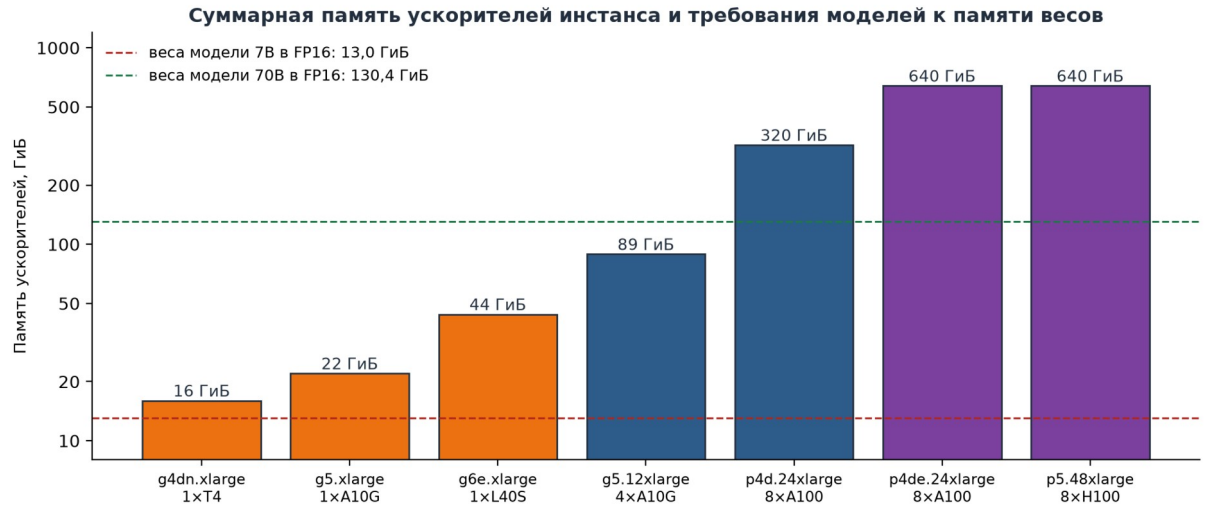


Рисунок 10.1 — Память ускорителей и требования моделей

10.2. Оценка памяти модели

Основной вопрос при выборе ускорителя — поместится ли модель в его память. Первое приближение даёт память весов: число параметров P, умноженное на размер одного параметра b.

Таблица 10.2 — Память весов (1 ГиБ = 2^{30} байт)

Формат	Байт на параметр	Модель 7 млрд	Модель 70 млрд
FP32	4	26,08 ГиБ	260,77 ГиБ
FP16, BF16	2	13,04 ГиБ	130,39 ГиБ

Формат	Байт на параметр	Модель 7 млрд	Модель 70 млрд
INT8	1	6,52 ГиБ	65,19 ГиБ
INT4	0,5	3,26 ГиБ	32,60 ГиБ

Пример расчёта для модели 7 млрд параметров в FP16: $7 \cdot 10^9 \times 2 = 14 \cdot 10^9$ байт; $14 \cdot 10^9 / 2^{30} \approx 13{,}04$ ГиБ. Обратите внимание на единицы: 14 ГБ (десятичных) — это 13,04 ГиБ (двоичных).

Этого недостаточно для работающей системы. При инференсе нужны буферы активаций и кэш ключей и значений механизма внимания; их объём зависит от длины контекста и размера пакета. При **полном обучении** со смешанной точностью и оптимизатором Adam на каждый параметр хранятся: веса FP16 (2 байта), градиенты FP16 (2 байта), копия весов FP32 (4 байта) и два момента Adam FP32 (4 + 4 байта) — всего **16 байт на параметр** без учёта активаций. Эта оценка приведена в работе ZeRO (Rajbhandari и др., 2020), на которой основаны DeepSpeed и FSDP.

QLoRA хранит замороженные веса в 4-битном формате и обучает лишь небольшие добавочные матрицы (адаптеры), поэтому требования к памяти близки к памяти 4-битных весов с запасом на адаптеры и активации. Именно это позволяет дообучать модели на одном ускорителе.

Замечание. Модель, веса которой не помещаются в один ускоритель, разбивается между несколькими (тензорный или конвейерный параллелизм). Суммарная память инстанса при этом необходима, но не достаточна: нужны поддержка разбиения в программном обеспечении и быстрая связь между ускорителями.

10.3. Разобранный пример выбора инстанса

Условие. Требуется дообучить методом QLoRA модель с 32 млрд параметров. Допущения задачи (раздел 15): $M = P \times 0{,}5 \times 1{,}5$.

1. Число параметров: $P = 32 \cdot 10^9$.
2. Память весов в 4 битах: $32 \cdot 10^9 \times 0{,}5 = 16 \cdot 10^9$ байт.
3. С запасом 1,5: $24 \cdot 10^9$ байт; в гибибайтах: $24 \cdot 10^9 / 2^{30} \approx 22{,}35$ ГиБ.
4. Кандидаты по возрастанию памяти: g4dn.xlarge (16) — мало; g5.xlarge (22) — мало, так как $22 < 22{,}35$; g6e.xlarge (44) — достаточно.
5. Ответ: g6e.xlarge, один ускоритель NVIDIA L40S; разбиение модели не требуется.

Пример показывает, почему важна точность в единицах: при расчёте в десятичных гигабайтах (24 ГБ) легко ошибочно выбрать g5.xlarge, память ускорителя которого в документации указана как 22 ГиБ.

10.4. Стоимость ускорителей и модели оплаты

Ускоренные инстансы — самые дорогие ресурсы облака, поэтому модель оплаты для них выбирается особенно тщательно (модели оплаты разобраны на занятии 3). Обучение, которое сохраняет контрольные точки и может быть возобновлено, выполняют на прерываемых инстансах: поставщик уведомляет об изъятии за две минуты, и задача продолжается с последней контрольной точки. Постоянная нагрузка инференса покрывается планами экономии. Цены ускоренных инстансов меняются: в июне 2025 года AWS снизила цены по требованию на инстансы P4d и P4de на 33 %, P5 — на 44 %. Поэтому в расчётах стоимости всегда указывается дата цены и цена сверяется с калькулятором стоимости.

11. Вычислительные ресурсы для обучения и применения моделей

Модели машинного обучения предъявляют к вычислительному слою особые требования: обучение — длительная задача с большим потреблением памяти ускорителя, применение — короткая задача с требованием к задержке. Эти две нагрузки размещают по-разному.

Этап	Что определяет выбор	Типичное решение
Подготовка данных	объём и параллелизм чтения	инстансы с быстрым локальным диском, пакетная обработка
Обучение	память ускорителя, межузловая сеть	ускоренные инстансы, прерываемые, контрольные точки
Подбор гиперпараметров	число независимых прогонов	очередь заданий, прерываемые инстансы
Применение (инференс)	задержка и число запросов	небольшой ускоритель или процессор, автомасштабирование
Периодический расчёт	объём и срок	пакетное применение вместо постоянной службы

Управляемый сервис обучения. Вместо запуска инстанса вручную задание описывается как задача: образ с кодом, данные во внешнем хранилище, тип и число инстансов, параметры. Сервис запускает ресурсы, выполняет обучение, сохраняет результат и **освобождает ресурсы**. Это важная деталь экономики: забытый инстанс с ускорителем стоит на порядок дороже забытого обычного.

Распределённое обучение. Когда модель или выборка не помещаются на один ускоритель, обучение распределяют: параллелизм по данным (каждый узел считает свою часть выборки и узлы обмениваются градиентами) или параллелизм по модели (слои модели разнесены по ускорителям). Эффективность распределения измеряется отношением полученного ускорения к числу узлов; она падает из-за обмена градиентами, поэтому у ускоренных инстансов важна пропускная способность межузловой сети.

Собственные ускорители. Помимо графических процессоров применяются специализированные чипы: одни рассчитаны на обучение, другие — на применение моделей. Их выбирают, когда стоимость единицы вычислений важнее универсальности, а используемая библиотека поддерживает такой чип.

Связь с практикой. Линия ML. Расчёт памяти модели из раздела выше даёт нижнюю границу требований к ускорителю. Решение о числе узлов принимается по той же логике, что и решение о числе реплик службы: измеряется эффективность, а не берётся «побольше на всякий случай».

12. От локальной ВМ к облаку



Рисунок 12.1 — Соответствие локальных понятий и сервисов AWS

Каждое действие лабораторной работы имеет облачный аналог. Настройка числа vCPU и памяти ВМ соответствует выбору типа инстанса; снимок ВМ — снимку тома EBS или собственному AMI; проброс порта SSH через NAT — публичному адресу инстанса, интернет-шлюзу и правилу группы безопасности; сценарий настройки после установки — пользовательским данным. Различие в том, что в облаке эти действия выполняются через программный интерфейс и потому воспроизводимы и автоматизируемы.

12.1. Методика сравнительного измерения производительности

Задание 3 требует сравнить время выполнения одной программы в двух средах. Результат такого сравнения корректен, только если различается одна-единственная величина — среда выполнения. Поэтому соблюдаются следующие правила.

1. **Одинаковая программа.** Исходный текст, компилятор, его версия, флаги оптимизации и тип сборки (Release) совпадают.
2. **Одинаковые входные данные.** Число сценариев, горизонт и начальное значение генератора совпадают; совпадение результата (VaR) подтверждает, что выполнялась одна и та же работа.
3. **Несколько запусков.** Время одного запуска подвержено случайным колебаниям: фоновые процессы, частота процессора, кэши. Выполняется не менее пяти запусков, в качестве оценки берётся **медиана**, устойчивая к единичным выбросам.
4. **Прогрев.** Первый запуск часто медленнее из-за загрузки программы и данных с диска; его можно исключить из статистики, указав это в отчёте.
5. **Сопоставимая нагрузка на хост.** Во время замера на хосте не выполняются ресурсоёмкие программы.

Относительная разница вычисляется как (T_ВМ − T_хост) / T_хост. Для вычислительной задачи без ввода-вывода при аппаратной виртуализации она обычно невелика; существенная разница указывает либо на нарушение методики, либо на конкуренцию за ресурсы, например на работу VirtualBox поверх Hyper-V.

Линия ML. Обучающиеся, изучающие дисциплину «Машинное обучение в облаке», в задании 5 дополнительно оценивают память модели, которую они обучают в этой дисциплине, и обосновывают выбор инстанса для её обучения и инференса. Задания с пометкой «Линия ML» рассчитаны на обучающихся, которые параллельно изучают дисциплину «Машинное обучение в облаке»: результат такого задания засчитывается в обеих дисциплинах и не требует двойной работы. Остальные обучающиеся выполняют то же задание с готовым эталонным компонентом, описанным в тексте; знание машинного обучения для этого не требуется.

13. Эталонная архитектура вычислительного слоя

Собранные вместе, разобранные элементы дают типовую схему вычислительного слоя приложения. Эта схема повторяется в большинстве решений и служит образцом для схемы, которую предстоит построить в лабораторной работе.

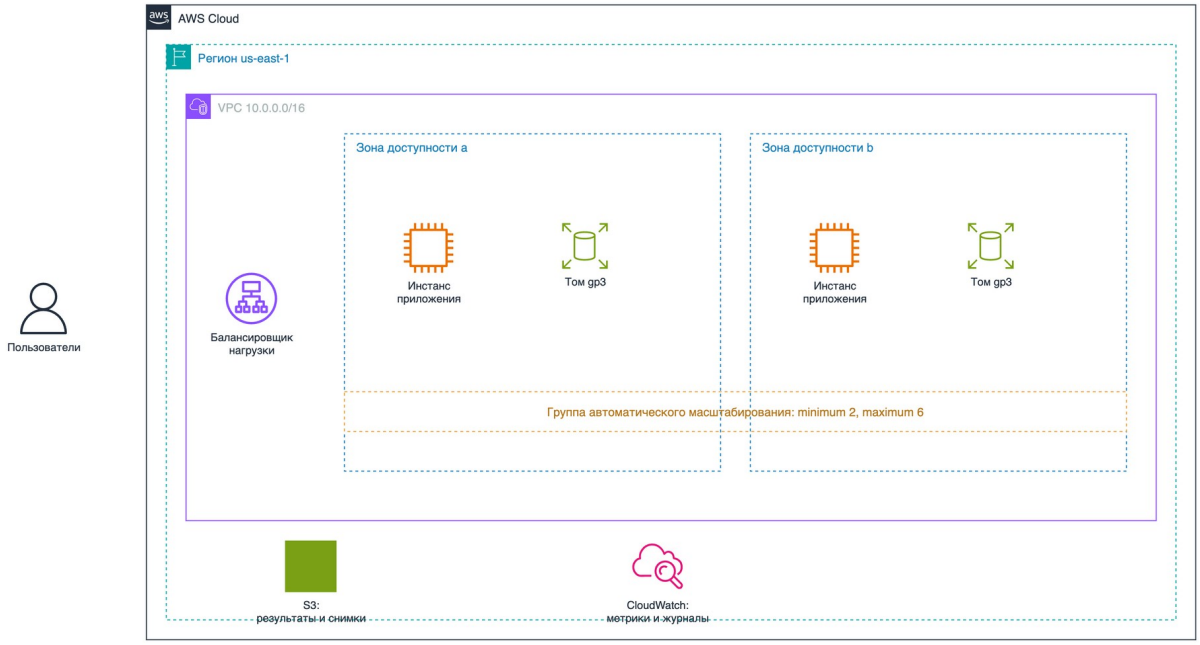


Рисунок 13.1 — Вычислительный слой приложения в облаке

1. запросы приходят на балансировщик нагрузки — единственную точку входа;
2. балансировщик распределяет их между инстансами в **двух зонах доступности**: отказ одной зоны не останавливает обслуживание;
3. инстансы создаются группой масштабирования по шаблону запуска и заменяются при отказе проверки;
4. состояние каждого инстанса хранится на томе gp3, общие результаты — в объектном хранилище;
5. метрики и журналы собираются в службе наблюдения, по ним работают политики масштабирования.

Из схемы сразу видны решения, которые придётся обосновать на защите проекта: сколько зон доступности, какой тип тома, какая проверка состояния, какая политика масштабирования, какой способ оплаты для базовой части реплик и для пиковой.

Элемент схемы	Отвечает за	Что произойдёт без него
Балансировщик	единая точка входа и распределение	клиенты обращаются к конкретным адресам
Две зоны доступности	переживание отказа зоны	отказ зоны останавливает службу
Группа масштабирования	желаемое число исправных реплик	замена отказавших вручную
Проверка состояния	исключение неисправных реплик	часть запросов уходит в неисправный узел
Объектное хранилище	общие данные и результаты	данные заперты на томе одного инстанса
Метрики и журналы	решения о масштабировании и разбор сбоев	решения принимаются вслепую

14. Разобранные примеры

Примеры разобраны полностью: условие, ход рассуждения, вычисления и вывод. Числа в них не «красивые» — они такие же, как в заданиях, чтобы порядок величин запоминался вместе со способом расчёта.

14.1. Счёт не замедлился, а копирование замедлилось вдвое

Условие. В виртуальной машине вычислительная задача выполняется на 3 % медленнее, чем на хосте, а копирование большого файла — почти вдвое медленнее. Почему разница такая?

1. Вычислительная задача состоит из обычных инструкций: они выполняются процессором напрямую, гипервизор в них не участвует.
2. Операции ввода-вывода требуют обращения к устройству, а устройство эмулируется: каждая такая операция вызывает выход из виртуальной машины и возврат в неё.
3. Стоимость одного выхода — тысячи тактов; при интенсивном вводе-выводе их число велико, и накладные расходы становятся заметными.
4. Средство исправления — паравиртуальные драйверы (virtio, гостевые дополнения): гость знает, что он гость, и обменивается с гипервизором блоками, а не отдельными обращениями к портам.
5. Проверка: после установки гостевых дополнений время копирования сокращается, а время счёта не меняется — оно и не было проблемой.

Вывод. Накладные расходы виртуализации определяются не «мощностью», а числом выходов из виртуальной машины. Отсюда правило: сначала смотреть на характер нагрузки, потом — на размер ресурса.

14.2. Выбор типа тома под заданные требования

Условие. Приложению нужно 12 000 операций в секунду и 400 МиБ/с при объёме данных 300 ГиБ. Какой тип тома подойдёт и во что это обойдётся по объёму?

1. Том gp3 даёт базовые 3000 операций и 125 МиБ/с независимо от объёма; дополнительная производительность заказывается отдельно, до 80 000 операций и 2000 МиБ/с.
2. Проверка ограничения gp3: на каждую операцию приходится не более 0,25 МиБ/с, значит для 400 МиБ/с нужно не менее 1600 операций — условие выполняется при 12 000.
3. Том gp2 даёт 3 операции на гибибайт: для 12 000 операций потребовался бы том 4000 ГиБ, то есть в 13 раз больше нужного объёма — только ради производительности.
4. Ответ: gp3 объёмом 300 ГиБ с заказанными 12 000 операций и 400 МиБ/с.
5. Том io2 понадобился бы при требовании выше 16 000 операций или при необходимости более высокой гарантии сохранности.

Вывод. Отделение производительности от объёма — главное практическое отличие gp3 от gp2. Оно же объясняет, почему «увеличить диск, чтобы ускорить» — приём из прошлого поколения томов.

14.3. Кредиты всплеска закончились

Условие. Том gp2 объёмом 100 ГиБ первые полчаса выдерживал 3000 операций в секунду, затем производительность упала до 300. Сколько длится всплеск и сколько восстанавливаются кредиты?

1. Базовая производительность: 3 операции на гибибайт × 100 = 300 операций в секунду.
2. Запас кредитов: 5,4 млн операций ввода-вывода.
3. Длительность всплеска при 3000 операций: 5 400 000 ÷ (3000 − 300) = 2000 с ≈ 33 минуты.
4. Восполнение полного запаса при простое: 5 400 000 ÷ (3 × 100) = 18 000 с = 5 часов.
5. Вывод для нагрузки, которой постоянно нужно 3000 операций: том gp2 должен быть не меньше 1000 ГиБ, либо нужен gp3 с заказанной производительностью.

Вывод. Кредитная модель даёт хорошие результаты на неравномерной нагрузке и плохие — на постоянной. Измерять нужно не пиковую производительность, а ту, что остаётся после исчерпания кредитов.

14.4. Когда обязательство окупается

Условие. Поставщик предлагает скидку 30 % за обязательство оплачивать ресурс в течение года. Какую долю времени ресурс должен работать, чтобы обязательство было выгоднее почасовой оплаты?

1. Обязательство оплачивается непрерывно: за год это 8760 часов по цене (1 − 0,30) от почасовой.
2. Почасовая оплата при доле использования u стоит u × 8760 часов по полной цене.
3. Равенство: 0,70 × 8760 = u × 8760, откуда u = 0,70.
4. Значит обязательство выгодно, если ресурс нужен более 70 % времени — примерно 17 часов в сутки.
5. Для нагрузки, работающей только в рабочие часы (примерно 24 % времени), выгоднее почасовая оплата, а ещё выгоднее — остановка ресурса на ночь и выходные.

Вывод. Точка окупаемости обязательства равна доле, дополняющей скидку: при скидке d выгода начинается при использовании (1 − d). Это правило применимо к любому плану экономии независимо от поставщика.

14.5. Выбор инстанса под применение модели

Условие. Нужно развернуть инференс модели с 13 млрд параметров в формате INT8 с запасом 1,5 на активации и кэш. Какой ускоритель подойдёт?

1. Память весов: $13 \cdot 10^9 \times 1$ байт = $13 \cdot 10^9$ байт.
2. С запасом: $13 \cdot 10^9 \times 1{,}5 = 19{,}5 \cdot 10^9$ байт.
3. Перевод в гибибайты: $19{,}5 \cdot 10^9 \div 2^{30} \approx 18{,}16$ ГиБ.
4. Кандидаты: g4dn.xlarge (16 ГиБ) — мало; g5.xlarge (22 ГиБ) — достаточно.
5. Ответ: g5.xlarge, один ускоритель NVIDIA A10G; разбиение модели между ускорителями не требуется.

Вывод. Ошибка в единицах измерения приводит к неверному выбору: 19,5 ГБ десятичных «влезают» в 22, но сравнивать нужно однородные величины — 18,16 ГиБ с 22 ГиБ.

15. Задачи для самопроверки

Задачи решаются без компьютера, по формулам и правилам занятия. Ответы приведены сразу после условий: сначала решите, потом сверьтесь.

15.1. Условия

1. Что такое выход из виртуальной машины и почему он важен для производительности?
2. Гипервизор работает поверх обычной операционной системы. К какому типу он относится?
3. Виртуальная машина должна быть доступна с другого компьютера сети. Какой режим сети выбрать?
4. Том gp3 объёмом 40 ГиБ. Какова его производительность без дополнительного заказа?
5. Сколько операций в секунду даст том gp2 объёмом 500 ГиБ?
6. Какой минимальный объём тома gp2 нужен, чтобы получить максимальные 16 000 операций?
7. Чем хранилище инстанса отличается от тома EBS по сохранности данных?
8. Что произойдёт с прерываемым инстансом при нехватке ёмкости и сколько времени есть у приложения?
9. Для чего служат пользовательские данные инстанса?
10. Модель 7 млрд параметров в FP16. Сколько памяти займут веса?
11. Сколько памяти на параметр требует полное обучение со смешанной точностью и оптимизатором Adam?
12. Почему при выборе инстанса нельзя ориентироваться только на суммарную память ускорителей?

15.2. Ответы

1. Передача управления от гостя гипервизору при попытке выполнить привилегированное действие. Каждый выход стоит тысячи тактов, поэтому число выходов определяет накладные расходы.
2. Ко второму: между оборудованием и гипервизором есть операционная система общего назначения.
3. Мостовой: машина получает адрес в той же сети, что и хост.
4. 3000 операций в секунду и 125 МиБ/с — базовые значения не зависят от объёма.
5. 3 × 500 = 1500 операций в секунду (базовая производительность).
6. 16 000 ÷ 3 ≈ 5334 ГиБ.
7. Хранилище инстанса исчезает при остановке инстанса; том EBS существует независимо и переживает остановку.
8. Инстанс будет изъят; уведомление приходит за две минуты, за которые нужно сохранить состояние.
9. Для автоматической настройки при первом запуске: сценарий выполняется после старта и делает среду воспроизводимой.
10. $7 \cdot 10^9 \times 2 = 14 \cdot 10^9$ байт ≈ 13,04 ГиБ.
11. 16 байт: веса FP16 (2) + градиенты FP16 (2) + копия весов FP32 (4) + два момента FP32 (4 + 4).
12. Модель должна быть разбита между ускорителями программно, и разбиение требует поддержки в библиотеке и быстрой связи между ускорителями.

16. Типичные ошибки и контрольные вопросы

16.1. Типичные ошибки

Ошибка	Последствие и исправление
Виртуализация выключена в прошивке	VirtualBox предлагает только 32-разрядные гостевые ОС или не запускает ВМ; включить Intel VT-x или AMD SVM в BIOS/UEFI
Образ другой архитектуры	ISO x86-64 не загружается на Apple Silicon; нужен образ ARM64
Проброс на занятый порт хоста	ВМ не стартует или SSH не подключается; выбрать свободный порт
Сравнение разных компиляторов	замер отражает качество кода, а не виртуализацию; собирать одним компилятором с одинаковыми флагами
Однократный замер	случайные колебания принимаются за эффект; делать не менее пяти запусков и брать медиану
Путаница ГБ и ГиБ	ошибка в 7,4 %; память ускорителей AWS указывает в ГиБ
Выбор по памяти весов без запаса	нехватка памяти при запуске; учитывать активации и кэш

16.2. Контрольные вопросы

1. Что такое виртуализация? Какие ресурсы вычислительной системы виртуализируются?
2. Сформулируйте три требования Попека — Голдберга к монитору виртуальных машин.
3. Какие инструкции называются чувствительными и почему архитектура x86 до появления аппаратной поддержки не допускала классической виртуализации методом перехвата и эмуляции?
4. Назовите три способа решения этой проблемы и укажите их недостатки.
5. Что добавляют к процессору технологии Intel VT-x и AMD-V? Что такое VM exit?
6. Зачем нужна аппаратная трансляция адресов второго уровня (EPT, NPT)?
7. Чем гипервизор первого типа отличается от гипервизора второго типа? Приведите примеры.
8. К какому типу относится KVM и почему этот вопрос обсуждается?
9. Перечислите режимы сетевого адаптера VirtualBox. Какой режим позволяет подключиться к ВМ по SSH с хоста, но не выпускает ВМ во внешнюю сеть?
10. Зачем при режиме NAT настраивается проброс порта? Что будет, если хост-порт уже занят?
11. Что такое снимок ВМ? Чем снимок отличается от резервной копии?
12. Из каких компонентов состоит система AWS Nitro? Какая функция осталась за гипервизором?
13. Чем инстанс .metal отличается от обычного инстанса того же семейства?
14. Что такое пользовательские данные (user data) инстанса и когда они выполняются?
15. Перечислите состояния жизненного цикла инстанса EC2. Оплачивается ли инстанс в состоянии stopped?
16. Сколько памяти занимают веса модели с 13 млрд параметров в формате FP16 и INT4? Ответ дайте в ГиБ.
17. Почему полное обучение модели требует около 16 байт памяти на параметр, а инференс — около 2?
18. Какой инстанс AWS содержит 8 ускорителей A100 по 80 ГиБ, а какой — по 40 ГиБ?
19. Почему для корректного измерения накладных расходов виртуализации необходимо использовать один и тот же компилятор и одинаковые параметры сборки?
20. Объясните, почему результат mc-var не зависит от числа потоков. Какую ошибку распараллеливания позволяет обнаружить эта проверка?
21. Чем том EBS отличается от локального диска инстанса? Что происходит с данными при остановке инстанса в каждом случае?

22. Какие базовые значения производительности включены в цену тома gp3 и какими двумя соотношениями ограничена настройка дополнительной производительности?

23. Можно ли заказать том gp3 объёмом 20 ГиБ с 20 000 операций в секунду? Ответ обоснуйте расчётом.

24. Как вычисляется базовая производительность тома gp2 и при каком размере тома она достигает предела?

25. Что такое кредиты всплеска тома gp2? Выведите формулу длительности всплеска и примените её к тому объёмом 334 ГиБ.

26. Почему у тома gp3 нет механизма всплесков и чем это удобно при измерениях?

27. Какие составляющие входят в счёт за вычислительный ресурс? Что продолжает тарифицироваться у остановленного инстанса?

28. Сформулируйте условие выгодности плана экономии через долю времени работы ресурса и выведите его.

29. Чем план экономии для вычислений отличается от плана для инстансов EC2? Какой из них даёт большую скидку и какой ценой?

30. Какие требования предъявляются к программе, работающей на прерываемых инстансах? Почему обучение модели подходит для них лучше, чем обслуживание запросов пользователей?

31. Что задают минимальное, желаемое и максимальное значения группы масштабирования? К чему приводит размещение группы в одной зоне доступности?

32. Оцените требуемое число инстансов при текущих четырёх инстансах, средней загрузке 88 % и цели 55 %.

33. Почему автоматическое масштабирование не защищает от мгновенного всплеска нагрузки? Чем защищаются?

34. По каким признакам выбирают между инстансом, задачей контейнера и функцией? Приведите по одному случаю, когда каждый вариант предпочтителен.

35. Что даёт управляемый сервис обучения по сравнению с ручным запуском инстанса с ускорителем?

17. Лабораторная работа № 4

Виртуальная машина, накладные расходы виртуализации и выбор вычислительного ресурса

17.1. Цель и порядок выполнения

Цель — получить практический навык настройки виртуальной машины и её сети, измерить накладные расходы виртуализации, освоить операции с инстансами EC2 и научиться обоснованно выбирать вычислительный ресурс для модели.

Работа состоит из шести заданий. Задания 1–3 выполняются в VirtualBox, задание 4 — с эмулятором AWS, задание 5 — расчётное, задание 6 — схема в draw.io. Параметры определяются номером варианта, который закреплён за обучающимся на весь курс.

17.2. Требования к отчёту

1. Отчёт в формате PDF: номер варианта, исходные данные, по каждому заданию — команды и полученный вывод в виде текста, ответы на вопросы заданий.
2. Таблица замеров задания 3 с медианами и расчётом накладных расходов.
3. Файл схемы .drawio и её изображение PNG.
4. Выводы: не менее трёх утверждений, вытекающих из полученных результатов.

17.3. Таблица индивидуальных параметров

Таблица 17.1 — Параметры вариантов

№	vCPU / ОЗУ, МБ / диск, ГБ	Порт SSH	Тип инстанса	Регион	Разрешённая сеть SSH	Модель для задания 6
1	2 / 3072 / 30	2201	t3.small	eu-central-1	203.0.113.0/28	7 млрд, инференс, FP16
2	4 / 4096 / 35	2202	m5.large	eu-west-1	203.0.113.16/28	13 млрд, инференс, FP16
3	2 / 2048 / 40	2203	t3.micro	eu-north-1	203.0.113.32/28	7 млрд, инференс, INT8
4	4 / 3072 / 25	2204	c5.xlarge	ap-northeast-1	203.0.113.48/28	70 млрд, инференс, INT4
5	2 / 4096 / 30	2205	t3.small	ap-southeast-1	203.0.113.64/28	8 млрд, инференс, BF16
6	4 / 2048 / 35	2206	m5.large	us-west-2	203.0.113.80/28	3 млрд, обучение
7	2 / 3072 / 40	2207	t3.micro	sa-east-1	203.0.113.96/28	7 млрд, QLoRA, INT4
8	4 / 4096 / 25	2208	c5.xlarge	ca-central-1	203.0.113.112/28	34 млрд, инференс, INT8
9	2 / 2048 / 30	2209	t3.small	eu-west-2	203.0.113.128/28	1,5 млрд, обучение
10	4 / 3072 / 35	2210	m5.large	ap-south-1	203.0.113.144/28	70 млрд, инференс, FP16
11	2 / 4096 / 40	2211	t3.micro	us-east-2	203.0.113.160/28	13 млрд, инференс, INT4
12	4 / 2048 / 25	2212	c5.xlarge	eu-south-1	203.0.113.176/28	7 млрд, обучение
13	2 / 3072 / 30	2213	t3.small	ap-southeast-2	203.0.113.192/28	32 млрд, инференс, BF16
14	4 / 4096 / 35	2214	m5.large	eu-west-3	203.0.113.208/28	14 млрд, QLoRA, INT4
15	2 / 2048 / 40	2215	t3.micro	us-west-1	203.0.113.224/28	70 млрд, QLoRA, INT4

Таблица 17.2 — Параметры заданий 5 и 7

№	Размер тома, ГиБ	Требуется операций/с	Часов работы в неделю	Инстансов сейчас	Средняя загрузка	Цель
1	60	2000	168	3	88 %	55 %
2	80	2500	60	4	110 %	55 %
3	100	3000	40	5	66 %	60 %

№	Размер тома, ГиБ	Требуется операций/с	Часов работы в неделю	Инстансов сейчас	Средняя загрузка	Цель
4	120	3500	100	2	44 %	50 %
5	140	4000	24	3	95 %	65 %
6	160	4500	120	4	120 %	55 %
7	60	5000	50	5	72 %	60 %
8	80	2000	168	2	58 %	50 %
9	100	2500	70	3	132 %	55 %
10	120	3000	45	4	39 %	60 %
11	140	3500	90	5	101 %	50 %
12	160	4000	30	2	84 %	65 %
13	60	4500	140	3	47 %	55 %
14	80	5000	55	4	117 %	60 %
15	100	2000	110	5	63 %	50 %

Модель для задания 6 указана в таблице 17.1 в последнем столбце.

17.4. Задания

Задание 1. Проверка аппаратной поддержки виртуализации

Выполните проверку по пункту 2.1 для своей ОС. Приведите вывод команды и модель процессора. Ответьте: какая технология аппаратной виртуализации используется вашим процессором (VT-x, AMD-V или средства ARM) и включена ли она.

Задание 2. Настройка виртуальной машины и её сети

1. Создайте или подготовьте ВМ Ubuntu 24.04 с параметрами своего варианта (vCPU, ОЗУ, диск). Выполните настройку командами `VBoxManage` и приведите их в отчёте.
2. Первый адаптер — NAT с пробросом порта хоста из таблицы 17.1 на порт 22 ВМ. Установите в ВМ сервер SSH (`sudo apt install -y openssh-server`) и подключитесь с хоста: `ssh -p <порт> <пользователь>@127.0.0.1`.
3. Второй адаптер — виртуальный адаптер хоста. Выведите адреса ВМ командой `ip -br addr` и подключитесь к ВМ по адресу сети host-only на порт 22 без проброса.
4. Создайте снимок `clean`, создайте в ВМ файл `/tmp/marker`, остановите ВМ, восстановите снимок и убедитесь, что файла нет.

Ответьте: почему во втором подключении проброс порта не нужен? Какой из двух способов доступа соответствует подключению к инстансу EC2 из-за пределов VPC, а какой — изнутри VPC?

Задание 3. Измерение накладных расходов виртуализации

В опорных файлах находится проект `bench` — оценка Value-at-Risk методом Монте-Карло (этот же проект используется на занятии 11). Программа печатает значение VaR и время параллельной и последовательной частей.

bench/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(mc_var VERSION 1.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Одноконфигурационные генераторы (Makefile, Ninja): по умолчанию собираем Release
if(NOT CMAKE_CONFIGURATION_TYPES AND NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Release CACHE STRING "Тип сборки" FORCE)
endif()

find_package(Threads REQUIRED)
add_executable(mc-var main.cpp)
target_link_libraries(mc-var PRIVATE Threads::Threads)
```

Сборка и запуск

```
# Linux, macOS и ВМ Ubuntu (генератор Makefiles, тип сборки Release по умолчанию)
cmake -S bench -B build
cmake --build build
./build/mc-var --threads 1

# Windows, Visual Studio (многоконфигурационный генератор)
cmake -S bench -B build
cmake --build build --config Release
.\build\Release\mc-var.exe --threads 1
```

1. Соберите `bench` на хосте и в ВМ. В ВМ установите компилятор: `sudo apt install -y build-essential cmake`.
2. На хосте и в ВМ выполните по пять запусков `mc-var --threads 1` и `mc-var --threads K`, где K — число vCPU вашей ВМ. Для каждой серии найдите медиану `total_ms`.
3. Рассчитайте накладные расходы: (T_ВМ − T_хост) / T_хост × 100 %.
4. Сравните значения `VaR99` на хосте и в ВМ.

Ответьте в отчёте: совпали ли значения VaR и почему это важно; каким компилятором собрана программа на хосте и в ВМ и как это влияет на корректность сравнения; почему при K потоках ускорение в ВМ не может превысить K, даже если процессор хоста имеет больше ядер.

> **Методическое требование.** Если на хосте используется другой компилятор (например, Visual Studio в Windows), для корректного сравнения повторите замер на хосте тем же компилятором, что и в ВМ, — например, в WSL (Ubuntu) с gcc. Если это невозможно, укажите смешение двух факторов в выводах.

Задание 4. Инстансы EC2 в эмуляторе

Запустите эмулятор и выполните команды (адрес эмулятора указывается параметром `--endpoint-url` или переменной `AWS_ENDPOINT_URL`, как на занятии 3):

1. получите характеристики своего типа инстанса и двух ускоренных типов командой `aws ec2 describe-instance-types --instance-types <тип> g5.xlarge p4d.24xlarge --query "InstanceTypes[].[InstanceType,VCpuInfo.DefaultVCpus,MemoryInfo.SizeInMiB,GpuInfo.Gpus[0].Name,GpuInfo.Gpus[0].MemoryInfo.SizeInMiB]" --output table`;
2. подготовьте файл user-data.sh (опорный файл), заменив NN номером варианта, и запустите инстанс своего типа с этими данными и тегом `Name=varNN`;
3. прочитайте пользовательские данные запущенного инстанса и декодируйте их из Base64;
4. проведите инстанс через состояния: остановка (`stop-instances`), запуск (`start-instances`), удаление (`terminate-instances`); после каждого шага выведите состояние.

Ответьте: сколько МиБ памяти GPU сообщил эмулятор для g5.xlarge и как это значение соотносится с 22 ГиБ в документации AWS?

Задание 5. Блочное хранилище варианта

По параметрам своего варианта (таблица 17.2) выполните расчёты и оформите их в отчёте:

1. вычислите базовую производительность тома gp2 такого же размера и укажите, достигает ли она требуемого числа операций;
2. вычислите длительность всплеска при полном балансе кредитов и время восстановления пустого баланса; сравните с таблицей примеров из раздела 6;
3. определите, какую дополнительную производительность нужно заказать для тома gp3, чтобы получить требуемое число операций, и наименьший допустимый размер такого тома;
4. вычислите наибольшую пропускную способность, доступную при этом числе операций;
5. ответьте: какой тип тома вы выберете для журнала операций сервиса, а какой — для архива котировок за прошлые годы, и почему.

Расчёты приводятся с формулами и промежуточными значениями, а не только с ответом.

Задание 6. Выбор инстанса для модели

Для модели своего варианта (таблица 17.1) рассчитайте требуемую память ускорителей и выберите инстанс из таблицы 10.1 с наименьшей суммарной памятью ускорителей, достаточной для модели. Правила расчёта (допущения задачи):

— инференс: $M = P \times b \times 1{,}2$, где b — байт на параметр по формату, 1,2 — запас на активации и буферы;

— полное обучение: $M = P \times 16$ байт;

— дообучение QLoRA: $M = P \times 0{,}5 \times 1{,}5$;

— 1 ГиБ = 2^{30} байт; рассматриваются инстансы g4dn.xlarge, g5.xlarge, g6e.xlarge, g5.12xlarge, p4d.24xlarge, p4de.24xlarge, p5.48xlarge.

Приведите расчёт, выбранный инстанс и ответьте: требуется ли для вашего случая разбиение модели между несколькими ускорителями; какой инстанс понадобился бы, если бы модель хранилась в формате FP32.

Задание 7. Способ оплаты и масштабирование

Служба варианта работает по графику из таблицы 17.2. Выполните расчёты:

1. определите долю времени работы службы за месяц (730 часов) и сравните её с порогом окупаемости плана экономии для вычислений (скидка 66 %) и плана для инстансов EC2 (скидка 72 %);
2. сформулируйте вывод: какой способ оплаты выгоден для этой службы и почему;
3. оцените требуемое число инстансов при текущем числе, средней загрузке и цели из таблицы; приведите расчёт с округлением вверх;
4. укажите, какие два параметра группы масштабирования вы зададите и почему именно такие;
5. ответьте: какую часть реплик допустимо разместить на прерываемых инстансах и при каком условии.

Задание 8. Архитектурная схема в draw.io

Постройте схему вычислительного слоя своего варианта: облако AWS, регион варианта, VPC 10.0.0.0/16, **две зоны доступности**, публичные подсети 10.0.1.0/24 и 10.0.2.0/24, интернет-шлюз, балансировщик нагрузки, группа масштабирования с инстансами своего типа, том gp3 у

каждого инстанса, объектное хранилище для результатов, служба метрик и журналов, группа безопасности, разрешающая вход по TCP 22 только из сети варианта, и администратор за пределами облака. На схеме подпишите: регион, адреса сетей, тип инстанса, размер и тип тома, границы группы масштабирования (minimum и maximum) и правило группы безопасности. Образец оформления — рисунок 13.1 и рисунок 17.1.

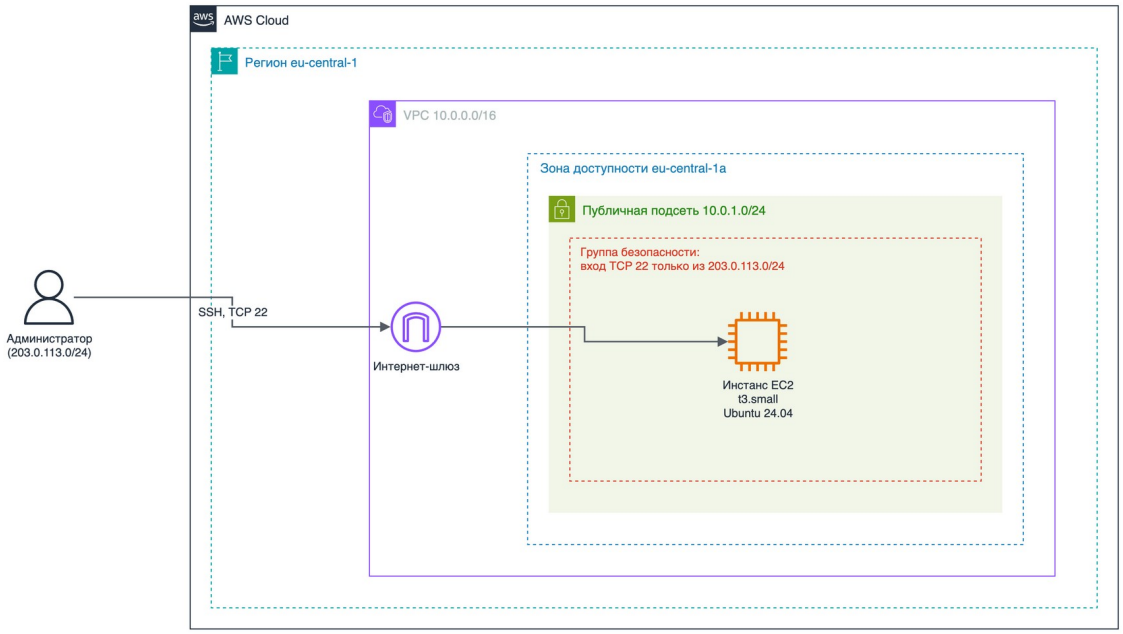


Рисунок 17.1 — Пример оформления схемы (вариант 1)

17.5. Задания повышенной сложности

Задания этого раздела выполняются по желанию и оцениваются дополнительно. Они не требуют новых средств: всё необходимое уже установлено и разобрано. Требуется другое — самостоятельно спланировать измерение или расчёт, выбрать способ проверки и объяснить полученный результат. Задание засчитывается только с приведённой методикой: что измерялось, чем, сколько раз и почему полученному числу можно верить.

Задание П1. Граница накладных расходов виртуализации. Постройте зависимость накладных расходов виртуализации от доли операций ввода-вывода в нагрузке. Возьмите три программы: чисто вычислительную (расчёт своего варианта), чисто вводную (копирование файла 1 ГиБ) и смешанную (расчёт с записью промежуточных результатов каждые 10 000 испытаний). Измерьте время каждой в хосте и в виртуальной машине, постройте таблицу и объясните, почему замедление различается. Оцените, при какой доле ввода-вывода накладные расходы превысят 25 %.

Задание П2. Стоимость владения при трёх моделях оплаты. Для инстанса своего варианта постройте график суммарной стоимости за год при трёх моделях оплаты (по требованию, план экономии со скидкой 30 %, прерываемые ресурсы со скидкой 70 % и потерей 5 % времени) как функции доли использования от 10 % до 100 % с шагом 10 %. Найдите точки пересечения и укажите, какая модель выгодна на каждом участке. График постройте средствами Python.

Задание П3. Выбор тома под заданный профиль нагрузки. Нагрузка состоит из двух фаз: 20 минут записи со скоростью 250 МиБ/с и 4 часа чтения со скоростью 40 МиБ/с. Подберите тип и параметры тома для трёх случаев: наименьшая стоимость; наименьшая задержка; запас на удвоение нагрузки. Для каждого случая приведите расчёт по документированным формулам и объясните, какое ограничение оказалось определяющим.

Порядок оценивания. Каждое задание оценивается отдельно. Условие зачёта одинаково: приведены методика, результаты и объяснение. Результат без объяснения, как и объяснение без результата, не засчитывается.

17.6. Критерии оценивания

Составляющая	Что оценивается	Доля
Задания 1–2	корректная настройка ВМ и сети, работающие подключения, восстановление снимка	18 %
Задание 3	пять запусков, медианы, расчёт, обоснованный ответ о корректности сравнения	20 %
Задание 4	все команды и ответы, прохождение жизненного цикла инстанса	12 %
Задание 5	расчёты по томам с формулами, верные значения, обоснованный выбор типа тома	16 %
Задание 6	верный расчёт памяти, обоснованный выбор инстанса	14 %
Задание 7	расчёт доли времени и порога, вывод о способе оплаты, оценка числа инстансов	12 %
Задание 8	полнота и корректность схемы, подписи, две зоны доступности	8 %

17.7. Этап сквозного проекта: каркас проекта и паспорт варианта

Этап открывает работу над сквозным проектом. Задача — завести единый каталог, в котором проект будет жить до конца курса, закрепить за собой числовые параметры варианта и убедиться, что генератор котировок воспроизводит эталонный ряд. Всё, что делается на последующих занятиях, добавляется в этот же каталог.

Что сделать.

1. Создайте каталог quantflow и в нём подкаталоги core, apps, services, docker, diagrams, docs, reports, data. Остальные подкаталоги добавляются по мере занятий; полная структура приведена в разделе Д.4 приложения Д. Каталог ведётся весь семестр и не пересоздаётся.
2. Заполните файл README.md паспортом варианта: номер варианта, инструмент, зерно, начальная цена, шаг, интервал, окна sma, ema и rsi (приложение Г, таблица Г.2), порты сервисов, регион своего варианта и тип инстанса, выбранный в задании 6. Ни одно из этих значений далее не записывается в текст программ: они передаются параметрами запуска или переменными окружения.
3. Реализуйте генератор котировок по разделу Г.2 приложения Г на выбранном языке либо соберите опорную реализацию. Выведите первые пять котировок своего варианта и сверьте их с таблицей Г.3.
4. Скопируйте файл своего варианта из каталога «Исходные тексты / Опорные файлы / Котировки по вариантам» в data/ и сверьте с ним построчно вывод своей реализации на 1000 котировок. Приведите команду сравнения и её результат.
5. Постройте первую версию общей схемы проекта diagrams/project.drawio: регион, виртуальная частная сеть, вычислительный инстанс выбранного типа и подключённый том. Это не схема задания 8, а схема всего проекта: она будет дополняться на каждом занятии.
6. Заведите журнал решений docs/decisions.md и внесите первую запись: почему выбран именно этот тип инстанса и тип тома. Запись содержит решение, две-три строки обоснования и числа из заданий 5—7.

Что появляется в проекте: README.md с паспортом варианта; реализация генератора и вывод первых пяти котировок; результат построчного сравнения с файлом котировок; diagrams/project.drawio и изображение; docs/decisions.md с первой записью.

Как проверяется на приёме:

— первые пять котировок совпадают с таблицей Г.3 для номера варианта;

— построчное сравнение с файлом котировок не даёт расхождений;

— в README.md заполнены все поля паспорта, значения совпадают с таблицей Г.2;

— схема проекта открывается и содержит подписанные регион, сеть, инстанс и том;

— в журнале решений есть запись с числами, а не с общими словами.

Время и оценка. Ориентировочное время выполнения этапа — 75 минут. Этап предъявляется вместе с отчётом по лабораторной работе и оценивается отдельно от неё: он входит в накопительную составляющую отметки и в итоговую защиту сквозного проекта.

18. Подготовка к итоговой аттестации

Ниже приведены вопросы итоговой аттестации, которые опираются на материал этого занятия, и план ответа на каждый: перечень того, что должно прозвучать. План — не текст ответа, а проверочный список: если какой-то пункт не удаётся раскрыть своими словами, к нему стоит вернуться в тексте занятия.

18.1. Гипервизоры первого и второго типов. Аппаратная поддержка виртуализации. Система Nitro

План ответа:

— определение виртуальной машины и роль гипервизора;

— первый тип: гипервизор работает на оборудовании; второй: поверх операционной системы; примеры и следствия для производительности;

— требования Попека — Голдберга и почему архитектура x86 им не удовлетворяла;

— аппаратная поддержка: режим гостя, выход из виртуальной машины и его цена;

— вложенные таблицы страниц: зачем и что они устраняют;

— вынос ввода-вывода на отдельные карты и почему это устраняет главный источник накладных расходов.

18.2. Семейства инстансов и ускорители. Оценка памяти для обучения и применения модели

План ответа:

— схема именования инстанса и что в ней закодировано;

— семейства: общего назначения, вычислительные, с увеличенной памятью, с ускорителями;

— понятие vCPU и почему сравнение по их числу между семействами некорректно;

— формула памяти весов: число параметров × байт на параметр; таблица форматов;

— 16 байт на параметр при полном обучении со смешанной точностью — из чего складываются;

— разница гигабайта и гибибайта и к какой ошибке она приводит при выборе инстанса.

18.3. Модели оплаты вычислительных ресурсов. Прерываемые инстансы и планы экономии

План ответа:

— оплата по требованию: когда она оправдана;

— план экономии: обязательство в долларах в час, скидка, срок;

— точка окупаемости обязательства: доля использования (1 − скидка);

— прерываемые ресурсы: скидка, уведомление об изъятии, требования к приложению;

— какие нагрузки допустимо переводить на прерываемые ресурсы, а какие нет;

— самый крупный источник экономии — размер ресурса по измерению, а не модель оплаты.

18.4. Хранилище данных инстанса: типы томов и их производительность

План ответа:

— локальное хранилище и сетевой том: сохранность данных при остановке;

— типы томов и их назначение; производительность, зависящая и не зависящая от объёма;

— базовые значения тома общего назначения и заказ дополнительной производительности;

— кредитная модель: базовая производительность, всплеск, восполнение;

— два ограничения (операции и пропускная способность) и правило: решает то, что нарушается первым.

19. Приложения

Приложение А. Чек-лист перед сдачей

Проверено	Что именно
☐	поддержка аппаратной виртуализации проверена, вывод приложен;
☐	виртуальная машина создана, режимы сети опробованы, проброс порта работает;
☐	измерение накладных расходов выполнено по методике: одна изменяемая величина, несколько повторов;
☐	результаты счёта и ввода-вывода приведены отдельно и объяснены;
☐	операции с инстансами выполнены через интерфейс командной строки, вывод приложен;
☐	тип тома выбран расчётом, проверены оба ограничения — операции и пропускная способность;
☐	модель оплаты выбрана с расчётом точки окупаемости;
☐	память ускорителя рассчитана в гибибайтах, инстанс выбран по документированным характеристикам;
☐	схема draw.io построена и подписана.

Приложение Б. Сводка команд для вычислительных ресурсов

Команда	Назначение
aws ec2 describe-instance-types --instance-types ТИП	характеристики типа инстанса
aws ec2 run-instances --image-id AMI --instance-type ТИП	запуск инстанса
aws ec2 describe-instances --query "Reservations[].Instances[].State.Name"	состояния инстансов
aws ec2 stop-instances --instance-ids ID	остановка (тома сохраняются)
aws ec2 terminate-instances --instance-ids ID	удаление инстанса
aws ec2 create-volume --size 60 --volume-type gp3 --iops 4000	создание тома с заказанной производительностью
aws ec2 attach-volume --volume-id ID --instance-id ID --device /dev/sdf	подключение тома
aws ec2 create-snapshot --volume-id ID --description "..."	снимок тома
aws ec2 describe-volumes --query "Volumes[].[VolumeId,Size,Iops,Throughput]"	перечень томов
aws autoscaling describe-auto-scaling-groups	группы масштабирования
aws ec2 describe-availability-zones --query "AvailabilityZones[].[ZoneName,ZoneId]"	зоны и их идентификаторы
aws service-quotas list-service-quotas --service-code ec2	квоты сервиса

Все команды выполняются против эмулятора: адрес задаётся параметром `--endpoint-url` или переменной окружения `AWS_ENDPOINT_URL`. Набор поддерживаемых эмулятором операций уже, чем в облаке: часть команд возвращает упрощённый ответ, и это отдельно отмечается в заданиях.

Приложение В. Глоссарий

Термин	Значение
Гипервизор	монитор виртуальных машин: программа (и оборудование), распределяющая ресурсы между ВМ
VM exit / VM entry	аппаратный переход из гостя в гипервизор и обратно
EPT, NPT	аппаратная трансляция адресов второго уровня для гостевой памяти
Паравиртуализация	гостевая ОС знает о виртуализации и явно вызывает гипервизор

Термин	Значение
virtio	стандарт паравиртуальных устройств в KVM
Nitro	аппаратно-программная платформа виртуализации EC2
AMI	образ машины, из которого запускается инстанс
User data	сценарий, выполняемый при первом запуске инстанса
ГиБ	гибибайт, 2^{30} байт; 1 ГБ = 10^9 байт ≈ 0,931 ГиБ
QLoRA	дообучение адаптеров низкого ранга поверх 4-битных весов

Приложение Г. Опорная библиотека: генератор котировок и индикаторы

Приложение описывает алгоритм генератора котировок, формулы индикаторов и контрольные значения по вариантам. Оно не зависит от языка программирования: приведены алгоритмы, формулы и числа, по которым реализация проверяется на совпадение. К этому приложению обращаются все последующие занятия курса, поэтому оно приведено здесь целиком.

Опорные тексты курса написаны на C++ и Python, однако собственные компоненты обучающийся вправе выполнять на другом языке программирования (см. раздел «Язык реализации» в тексте любого занятия начиная с пятого). Чтобы работу на любом языке можно было принять, результаты вычислений должны совпадать с эталонными. Настоящий документ задаёт всё, от чего эти результаты зависят: алгоритм генератора котировок, формулы индикаторов, порядок вычислений и контрольные значения.

Документ выдаётся обучающимся. Он не содержит готового кода: приведены алгоритмы, формулы и числа, по которым реализация проверяется.

Г.1. Соглашения о типах и единицах

— Цена хранится и передаётся целым числом в копейках. Величины с плавающей точкой для цены не применяются: они дают расхождения в последних знаках и делают сверку невозможной.

— Метка времени — целое число миллисекунд от 1970-01-01T00:00:00Z.

— Объём — целое неотрицательное число.

— Состояние генератора — 64-разрядное беззнаковое целое. Все операции выполняются по модулю 2^{64} с отбрасыванием переполнения.

— Индикаторы вычисляются над ценами в рублях, то есть над значением price_cents, делённым на 100, в формате двойной точности (IEEE 754 binary64).

Важно. В языках, где 64-разрядное целое знаковое (Java, C#, Kotlin), сдвиг вправо выполняется беззнаковым оператором, а остаток от деления — беззнаковой операцией. В языках с целыми произвольной точности (Python) после каждой операции применяется маска 0xFFFFFFFFFFFFFFFF. В JavaScript обычный тип number непригоден — требуется BigInt.

Г.2. Генератор котировок

Генератор построен на xorshift64* (S. Vigna, 2014) — сдвиги 12, 25, 27 и умножение на постоянную 2685821657736338717. Он выбран за две особенности: полностью определяется целочисленными операциями и воспроизводится побитово в любом языке.

Таблица Г.1 — Параметры генератора

Параметр	Смысл
symbol	обозначение инструмента варианта

Параметр	Смысл
seed	начальное состояние; если равно нулю, берётся 0x9E3779B97F4A7C15
base_cents	начальная цена в копейках
step_cents	наибольшее изменение цены за шаг, в копейках
interval_ms	интервал между котировками, миллисекунды

Начало отсчёта времени: kStartMs = 1767225600000 (2026-01-01T00:00:00Z).

Очередное 64-разрядное число:

```
state ^= state >> 12;            // беззнаковый сдвиг
state ^= state << 25;            // по модулю 2^64
state ^= state >> 27;
return state * 2685821657736338717;      // по модулю 2^64
```

Очередная котировка (index — порядковый номер, начиная с нуля):

```
span   = 2 * step_cents + 1;
delta  = (next_u64() mod span) - step_cents;   // остаток беззнаковый
price  = price + delta;
if (price < 1) price = 1;
volume = 100 + (next_u64() mod 900);
tick   = { symbol, kStartMs + index * interval_ms, price, volume };
index  = index + 1;
```

Внимание. На каждую котировку генератор обращается к next_u64 дважды: первое число даёт изменение цены, второе — объём. Порядок обращений менять нельзя: он определяет всю дальнейшую последовательность.

Г.3. Индикаторы

Ниже p — ряд цен в рублях, n — окно или период. Порядок операций важен: он влияет на последние знаки результата.

— Скользящая средняя sma(p, n). Среднее арифметическое каждых n подряд идущих значений. Длина результата — len(p) − n + 1. В ответе сервиса приводится последнее значение.

— Экспоненциальная средняя ema(p, n). Коэффициент α = 2 / (n + 1); начальное значение — sma первых n точек; далее $ema_i = \alpha \cdot p_i + (1 - \alpha) \cdot ema_{i-1}$ для i от n до конца ряда.

— Индекс относительной силы rsi(p, n) по Уайлдеру. Первое сглаживание: средний рост и среднее падение по первым n приращениям. Далее для каждого следующего приращения: gain = (gain·(n − 1) + max(d, 0)) / n, loss = (loss·(n − 1) + max(−d, 0)) / n. Результат: 100 − 100 / (1 + gain / loss); если loss = 0, результат равен 100.

— Логарифмические доходности $r_i = \ln(p_i / p_{i-1})$, i от 1 до конца ряда.

— Волатильность — выборочное среднее квадратическое отклонение логарифмических доходностей, то есть с делением на (m − 1), где m — число доходностей.

— Максимальная просадка — наибольшее относительное падение от предшествующего максимума: проход по ряду с обновлением максимума и вычислением (peak − p_i) / peak.

Типичная ошибка — генеральная дисперсия вместо выборочной. Делитель равен m − 1, а не m. Это самая частая причина расхождения волатильности в четвёртом знаке.

Г.4. Параметры вариантов

Параметры закреплены за обучающимся на весь курс и одинаковы во всех занятиях.

Таблица Г.2 — Параметры генератора и окна индикаторов по вариантам

№	Инструмент	Зерно	Цена, коп.	Шаг, коп.	Интервал, мс	sma	ema	rsi
1	ALFA	1037	5750	10	500	6	11	9
2	BRAVO	1074	6500	8	1000	7	14	14
3	CHARLIE	1111	7250	6	250	8	17	21
4	DELTA	1148	8000	4	500	9	20	7
5	ECHO	1185	8750	11	1000	10	10	9
6	FOXTROT	1222	9500	9	250	11	13	14
7	GOLF	1259	10250	7	500	12	16	21
8	HOTEL	1296	11000	5	1000	13	19	7
9	INDIA	1333	11750	3	250	14	9	9
10	JULIETT	1370	12500	10	500	15	12	14
11	KILO	1407	13250	8	1000	5	15	21
12	LIMA	1444	14000	6	250	6	18	7
13	MIKE	1481	14750	4	500	7	8	9
14	NOVEMBER	1518	15500	11	1000	8	11	14
15	OSCAR	1555	16250	9	250	9	14	21

Г.5. Контрольные векторы: первые пять котировок

Первая проверка реализации генератора: совпадение первых пяти котировок. Если они совпали, совпадёт и весь ряд.

Таблица Г.3 — Первые пять котировок (цена в копейках / объём)

№	Инструмент	1-я	2-я	3-я	4-я	5-я
1	ALFA	5745 / 858	5751 / 217	5748 / 944	5752 / 216	5751 / 678
2	BRAVO	6499 / 409	6496 / 987	6495 / 413	6502 / 320	6506 / 366
3	CHARLIE	7245 / 771	7244 / 498	7245 / 305	7247 / 874	7244 / 848
4	DELTA	7996 / 820	7993 / 776	7989 / 915	7985 / 420	7982 / 567
5	ECHO	8751 / 863	8749 / 849	8745 / 228	8754 / 503	8749 / 387
6	FOXTROT	9505 / 575	9509 / 131	9509 / 749	9508 / 267	9503 / 917
7	GOLF	10246 / 991	10241 / 370	10238 / 830	10237 / 592	10234 / 231
8	HOTEL	11000 / 457	10999 / 901	10996 / 374	10995 / 695	11000 / 842
9	INDIA	11749 / 163	11749 / 492	11752 / 505	11752 / 511	11750 / 895
10	JULIETT	12496 / 701	12489 / 778	12488 / 392	12498 / 420	12504 / 326
11	KILO	13255 / 107	13254 / 583	13248 / 771	13249 / 280	13242 / 567
12	LIMA	14002 / 523	14008 / 429	14005 / 169	14003 / 602	14004 / 330
13	MIKE	14747 / 119	14750 / 516	14747 / 161	14751 / 779	14747 / 919
14	NOVEMBER	15510 / 783	15518 / 546	15507 / 155	15501 / 453	15497 / 646
15	OSCAR	16244 / 630	16242 / 576	16244 / 838	16236 / 648	16232 / 719

Метка времени первой котировки во всех вариантах равна 1767225600000; каждая следующая больше предыдущей на interval_ms.

Г.6. Контрольные значения индикаторов

Значения получены по ряду из 1000 котировок с параметрами своего варианта. Допустимое расхождение при сверке: для волатильности и просадки — шесть знаков после запятой, для остальных величин — четыре.

Таблица Г.4 — Значения при длине ряда 1000

№	Инструмент	last	sma	ema	rsi	волатильность	просадка
1	ALFA	55,11	55,1917	55,2259	27,5393	0,001069	0,060998
2	BRAVO	65,26	65,2100	65,1893	61,6446	0,000768	0,021872
3	CHARLIE	72,76	72,7925	72,7924	48,1356	0,000513	0,016553
4	DELTA	79,10	79,0922	79,0641	60,3584	0,000323	0,018009
5	ECHO	88,20	88,4350	88,3940	15,8516	0,000772	0,024358
6	FOXTROT	95,94	95,9136	95,9001	56,3273	0,000570	0,014841
7	GOLF	102,58	102,6083	102,5932	48,5900	0,000424	0,015490
8	HOTEL	109,48	109,6223	109,5858	17,4535	0,000290	0,010806
9	INDIA	116,96	116,9286	116,9445	65,3128	0,000172	0,008666
10	JULIETT	123,19	123,3573	123,3020	41,5594	0,000486	0,023141
11	KILO	130,76	130,7220	130,7083	52,1004	0,000377	0,017039
12	LIMA	139,16	139,1267	139,1308	60,3804	0,000266	0,016284
13	MIKE	146,89	146,8800	146,8801	54,6339	0,000175	0,009246
14	NOVEMBER	152,19	152,1375	152,1462	52,3381	0,000444	0,021072
15	OSCAR	162,91	162,9422	162,9247	43,5233	0,000341	0,014447

Г.7. Готовые файлы котировок

В каталоге «Исходные тексты / Опорные файлы / Котировки по вариантам» настоящего занятия лежат файлы ticks_var01.csv … ticks_var15.csv: по 1000 котировок, столбцы symbol, ts_ms, price_cents, volume, разделитель — запятая, первая строка — заголовок.

Файлы служат двум целям: сверить собственную реализацию генератора и, если задание генератора не ставится, получить входные данные без его реализации. Ряды длиннее 1000 котировок (например, 500 000 в занятии по форматам данных) файлами не выдаются — они порождаются алгоритмом раздела 3 и являются продолжением того же ряда.

Г.8. Как проверить свою реализацию

— Сравнить первые пять котировок с таблицей Г.3 — проверяется генератор.

— Сравнить построчно весь файл ticks_varNN.csv с собственным выводом — проверяется генератор на длинном ряду, в том числе ограничение снизу «цена не меньше одной копейки».

— Сравнить шесть значений из таблицы Г.4 — проверяются индикаторы.

— Если сошлись котировки, но не сошлись индикаторы, ошибка в формулах; если не сошлись котировки — в целочисленной арифметике: знаковый сдвиг вместо беззнакового, знаковый остаток от деления или потеря старших разрядов при умножении.

Приложение Д. Сквозной проект курса: состав, порядок ведения и защита

Приложение описывает сквозной проект, работа над которым начинается на этом занятии и продолжается до конца курса. Здесь приведены состав проекта, карта его развития по занятиям, структура каталогов, правила ведения, порядок сдачи и требования к итоговой защите. Задание очередного этапа приведено в каждом занятии — в последнем подразделе лабораторной работы; на занятиях 14 и 15 — в последнем подразделе практической части.

Д.1. Что представляет собой проект

Сквозной проект курса — платформа рыночной аналитики: набор связанных сервисов, которые принимают поток котировок, вычисляют по нему показатели, отдают результат по

сетевым интерфейсам, хранят данные в объектном хранилище и таблице, защищены, собираются и публикуются автоматически и разворачиваются одной командой.

Проект не выполняется «в конце семестра» отдельной работой. Каждое занятие, начиная с четвёртого, добавляет к нему очередную часть, и эта часть предъявляется вместе с отчётом по лабораторной работе. Занятия 1—3 были подготовительными: на них осваивались система сборки, рабочее место и эмулятор программного интерфейса облака.

Все работы выполняются локально, без учётной записи облачного провайдера: обращения к облачным сервисам направляются в эмулятор программного интерфейса. Сервисы, которые невозможно воспроизвести локально, входят в проект в виде архитектурных схем и расчётов.

Индивидуальность. У каждого обучающегося свой вариант: инструмент, параметры генератора котировок, окна индикаторов, порты сервисов, регион и тип инстанса. Параметры приведены в таблице Г.2 приложения Г и закрепляются на весь курс. Числовые результаты воспроизводимы, поэтому преподаватель располагает эталонными значениями по каждому варианту.

Язык реализации. Опорные тексты написаны на C++ и Python. Собственные компоненты допускается выполнять на другом языке при условиях, перечисленных во врезке «Язык реализации». Выбранный язык не меняется до конца курса: проект собирается из компонентов всех занятий.

Д.2. Составные части проекта

Таблица Д.1 — Компоненты проекта

Компонент	Язык опоры	Назначение	Занятия
quant-core	C++	библиотека статистики и индикаторов, генератор котировок	4, 5, 11
tick-feed	C++	сервер потока котировок по TCP и клиент	6, 10
indicators-api	C++	сервис показателей; затем защищённое соединение, токены, метрики	8, 10, 12—15
quote-service	C++	сервис потоковых вызовов	9, 10
risk-model	Python	компонент модели с контрактом: прогноз и оценка аномальности	8, 10, 13
конвейеры данных	Python	озеро данных, поток, очередь, обработчик событий	7, 10, 11
инфраструктура	HCL, YAML	композиция, описание инфраструктуры, манифесты, конвейер	10, 13—15

Столбец «Язык опоры» указывает язык опорной реализации, а не обязательный язык выполнения.

Д.3. Карта проекта и трудоёмкость

Таблица Д.2 показывает, что добавляется к проекту на каждом занятии и сколько времени это занимает. Столбец «Лабораторная работа» — оценка времени на обязательные задания работы без заданий повышенной сложности; столбец «Этап проекта» — время на сборку результата в проект.

Таблица Д.2 — Этапы проекта и оценка трудоёмкости, часов

Зан.	Что добавляется в проект	Лабораторная работа	Этап проекта	Всего
4	каркас проекта и паспорт варианта	2,3	1,25	3,55
5	ядро проекта в контейнере	3	0,67	3,67
6	поток котировок и сеть проекта	2,5	0,67	3,17
7	хранилище данных проекта	2,5	0,75	3,25

Зан.	Что добавляется в проект	Лабораторная работа	Этап проекта	Всего
8	сервис показателей	2,5	0,83	3,33
9	потоковый интерфейс проекта	2,5	0,67	3,17
10	сборка композиции проекта	3,5	1,17	4,67
11	согласованность и расчёт риска	2	0,67	2,67
12	защита контура проекта	3,5	0,83	4,33
13	конвейер выпуска проекта	2	0,67	2,67
14	инфраструктура проекта в виде кода	2	0,83	2,83
15	кластер, наблюдаемость и сборка к защите	2,5	1,5	4
Всего		30,8	10,5	41,3

Суммарная трудоёмкость занятий 4—15 составляет 41,3 часа. Часть работы выполняется в аудитории: по регламенту на разбор и начало заданий отведено 15—20 минут каждого занятия, что даёт около 3,5 часа. На самостоятельную работу остаётся около 38 часов при 36 часах, отведённых учебным планом. Резерв создаётся двухнедельным сроком сдачи работ начиная с шестой: недельная нагрузка равна одной работе, а две недели дают запас на совмещение с письменными работами.

Д.4. Структура каталогов

Проект ведётся в одном каталоге. Опорные файлы каждого занятия распаковываются в него и дополняют уже имеющиеся, а не заменяют их.

```
quantflow/
├── README.md                паспорт варианта и порядок запуска
├── CMakeLists.txt           корневая сборка проекта
├── core/                    библиотека расчётов и генератор котировок
├── apps/                    quant-cli, tick-feed, serialize, mc-var
├── services/                indicators-api, quote-service, risk-model
├── schemas/                 tick.proto, openapi.yaml
├── docker/                  описания образов
├── deploy/                  настройка шлюза
├── jobs/                    разовые задачи и генератор потока
├── messaging/               очередь, тема, поток, обработчик событий
├── lake/                    сценарии загрузки и локальные копии зон озера
├── terraform/               описание инфраструктуры кодом
├── k8s/                     манифесты кластера
├── monitoring/              метрики и обнаружение дрейфа
├── .github/workflows/       конвейер сборки и проверки
├── diagrams/                project.drawio и изображения
├── docs/                    decisions, network, storage, security,
│                            measurements, cost, risks, review
├── data/                    котировки варианта
└── reports/                 отчёты по занятиям в формате PDF
```

Д.5. Правила ведения проекта

— Один каталог на весь курс. Результаты занятий не разносятся по отдельным папкам и не пересоздаются заново: каждая работа продолжает предыдущую.

— Паспорт варианта в README.md. Все программы берут значения варианта из параметров запуска или переменных окружения, а не из текста программы.

— Воспроизводимость. Любой результат должен получаться повторно: команда запуска записана, входные данные заданы вариантом, числовые значения совпадают с эталонными из приложения Г.

— Схема накапливается. Файл diagrams/project.drawio дополняется на каждом занятии и к итоговому занятию содержит всю архитектуру. Схема задания и схема проекта — разные файлы.

— Измерения записываются сразу, в docs/measurements.md: размеры образов, трафик, время ответа, число вызовов, стоимость запросов. Восстановить их в конце семестра невозможно.

— Журнал решений docs/decisions.md: на каждом занятии — запись о принятом решении, две-три строки обоснования и числа. На защите требуется объяснить не «как сделано», а «почему так».

— Секреты не попадают в проект: ключи, пароли и токены задаются переменными окружения и не сохраняются ни в образах, ни в описаниях, ни в отчётах.

Д.6. Порядок сдачи и оценивания

Этап проекта сдаётся вместе с отчётом по лабораторной работе того же занятия. Приём занимает 3—5 минут: обучающийся показывает, что проект запускается, что числа совпадают с эталонными и что схема и журнал дополнены. Занятия 14 и 15 лабораторной работы не содержат: их этапы принимаются на занятии 16 вместе с итоговым отчётом по проекту. Отдельной сдачи проекта в конце семестра не предусмотрено: к итоговому занятию проект уже собран, и защищается то, что накоплено.

Этап оценивается отдельно от лабораторной работы. Лабораторные работы дают 45 % отметки, письменные работы — 20 %, архитектурные схемы и этапы проекта накопительно — 15 %, защита — 20 %. Невыполненный этап осложняет следующий: каждый опирается на предыдущий.

Д.7. Итоговая защита

Защита проводится на занятии 16. Порядок для каждого обучающегося: доклад по схеме, демонстрация работающей системы, ответы на вопросы. Предъявляется следующее.

— Итоговая схема архитектуры своего варианта: сервисы, сети, хранилища, защита, наблюдение; подписаны диапазоны адресов, порты, протоколы и форматы данных.

— Работающая система: композиция сервисов либо развёртывание в кластере, поднимаемые одной командой по описанию, без ручных действий.

— docs/review.md — разбор по шести направлениям анализа архитектуры: что сделано, чего нет и почему.

— docs/cost.md — расчёт месячной стоимости по составляющим и обоснование выбора способа оплаты.

— docs/risks.md — не менее трёх принятых рисков с обоснованием.

— docs/measurements.md — сводка измерений за курс; docs/decisions.md — журнал решений.

Проект не принимается, если система не поднимается по описанию, схема не соответствует показанному или расчёт стоимости отсутствует.

Таблица Д.3 — Критерии оценки защиты

Составляющая	Что оценивается	Доля
Схема архитектуры	полнота, подписи, соответствие варианту	20 %
Демонстрация	система поднимается и работает, показан путь запроса	25 %
Разбор по направлениям	содержательные ответы по каждому из шести	20 %
Расчёт стоимости	составляющие, тарифы, выбор способа оплаты	15 %
Принятые риски	названы прямо и обоснованы	10 %
Ответы на вопросы	обоснование решений по существу	10 %

Д.8. Чек-лист готовности проекта

— □ каталог проекта единый, опорные файлы всех занятий распакованы в него;

— □ README.md содержит паспорт варианта, порядок запуска и ссылки на документы проекта;

— □ система поднимается одной командой и отвечает на запрос;

— □ показатели совпадают с таблицей Г.4 приложения Г;

— □ diagrams/project.drawio дополнялся на каждом занятии, итоговая схема построена;

— □ docs/measurements.md заполнен по всем занятиям, docs/decisions.md содержит не менее десяти записей;

— □ docs/review.md заполнен, включая пункты «чего нет и почему»;

— □ docs/cost.md посчитан по составляющим и согласован со схемой;

— □ docs/risks.md содержит не менее трёх рисков с обоснованием;

— □ секретов нет ни в образах, ни в описаниях, ни в отчётах;

— □ ресурсы, созданные описанием инфраструктуры, удаляются полностью;

— □ отчёты по всем занятиям сохранены в каталоге reports.